



兰州工业学院

实习（实训）报告

(2023 —2024 学年第 2 学期)

实习（实训）名称 单片机应用系统

设计及实现

学 院 电子信息工程学院

专业班级 专升本电信 23

姓 名 姜 勇

学 号 ZB2305010116

指导教师 吴红娟、张艳茹

起止日期 2024. 6. 17--2024. 6. 28

目录

一、功能要求分析.....	2
二、 总体设计方案.....	2
2.1 系统架构：	2
2.2 硬件选型：	3
2.3 软件设计：	4
三、 模块设计方案.....	4
3.1 显示功能模块：	4
3.2 键盘功能模块：	8
3.3 闹铃、提醒功能模块：	10
3.4 温度测量功能模块：	11
3.5 系统主控电路原理图设计，系统外围电路原理图设计.....	15
四、程序设计与调试.....	15
4.2 DS18B20 模块程序设计、调试.....	27
4.3 计时功能模块程序设计、调试.....	32
4.4 按键模块程序设计、调试.....	35
4.5 闹钟和整点报时模块程序设计、调试.....	47
4.6 串口模块设计、实现.....	50
4.7 利用 Proteus 软件实现系统仿真、系统软硬件调试、实现系统要求功能.....	57
4.8 主函数设计.....	58
实习（实训）总结.....	63

实习（实训）时间	2024 年 6 月 17 日-2024 年 6 月 28 日
实习（实训）地点	信息与通信工程基础实验室（微机系统实验室）
实习（实训）目的 单片机系统设计及实现是电子信息工程专业（专升本）一个重要的实践性环节。通过对 51 系列单片机的硬件设计、软件编程、仿真环节的实现，加深对单片机知识的理解和掌握，掌握小型单片机应用系统的设计和调试方法，提高学生的单片机应用能力，为后续课程学习和今后从事电子产品的设计、调试等工作打下必要的基础。	
实习（实训）日志	
日期：6 月 17 日 实习（实训）内容： 一、功能要求分析 年、月、日、时、分、秒的计时功能。 时间显示在 LCD 液晶上。 时间可通过键盘进行设定和调整。 具备闹铃设定和提醒功能。 可采集周围环境温度并显示。 能用上位机串口助手远程配置时间和闹钟值。 当环境温度高于或低于给定上、下限阈值时，系统将报警，且有 LED 同步闪烁（拓展任务）。 二、总体设计方案 2.1 系统架构： • 采用 51 系列单片机作为核心控制器 • 外接 LCD 液晶显示模块、键盘模块、温度传感器模块、串口模块、LED 报警模块等 • 系统通过单片机的 I/O 口与各模块进行通信和控制	

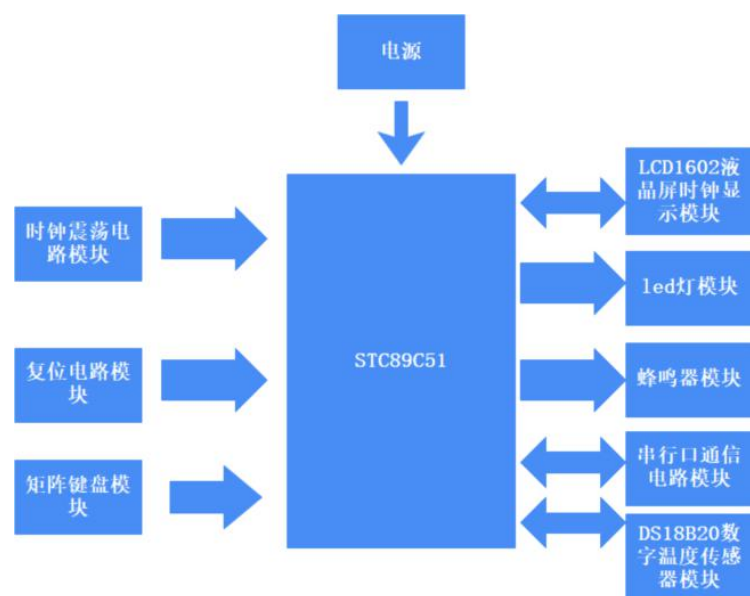


图 1 基于单片机系统的电子时钟基本结构框图

2.2 硬件选型:

- 单片机: 选择 STC89C52 单片机
- LCD 液晶显示模块: 选择 1602 液晶模块
- 键盘模块: 选择 4×4 矩阵键盘
- 温度传感器模块: 选择 DS18B20 数字温度传感器
- 串口模块: 使用单片机的内部 UART
- LED 报警模块: 选择普通发光二极管

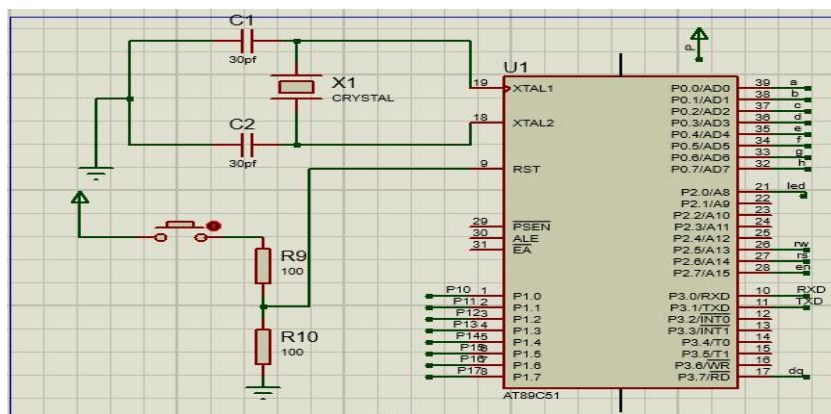


图 2 单片机的最小系统及晶振电路、复位电路

2.3 软件设计:

- 采用 C 语言进行编程, 实现系统的各项功能
- 软件设计包括主程序、中断服务程序、LCD 显示程序、键盘扫描程序、温度采集程序、闹铃程序、串口通信程序等

日期: 6 月 18 日

实习(实训)内容:

三、模块设计方案

3.1 显示功能模块:

- 采用 1602 液晶模块, 通过单片机的 I/O 口进行控制
- 显示内容包括年、月、日、时、分、秒、温度等信息

时钟需要 24 小时制, 分和秒需要 60 进制计数。当秒达到 60 时, 清零并使分增加 1; 当分达到 60 时, 清零并使时增加 1; 当时达到 24 时, 清零, 并使天增加 1, 当天达到 31 时, 清零并使月增加 1, 当月达到 12 时, 清零并使年增加 1。需要定时器来控制时间的更新。时间显示在 LCD 液晶上。

将定时器 0 设置为模式 1, 即 16 位定时器。初始值设置为 50ms, 每 20 次中断一次, 实现 1s 中断效果。

1602 液晶也叫 1602 字符型液晶, 它能显示 2 行字符信息, 每行又能显示 16 个字符。它是一种专门用来显示字母、数字、符号的点阵型液晶模块。它是由若干个 5x7 或者 5x10 的点阵字符位组成, 每个点阵字符位都可以用显示一个字符, 每位之间有一个点距的间隔, 每行之间也有间隔, 起到了字符间距和行间距的作用。

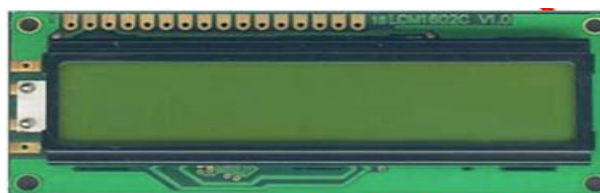
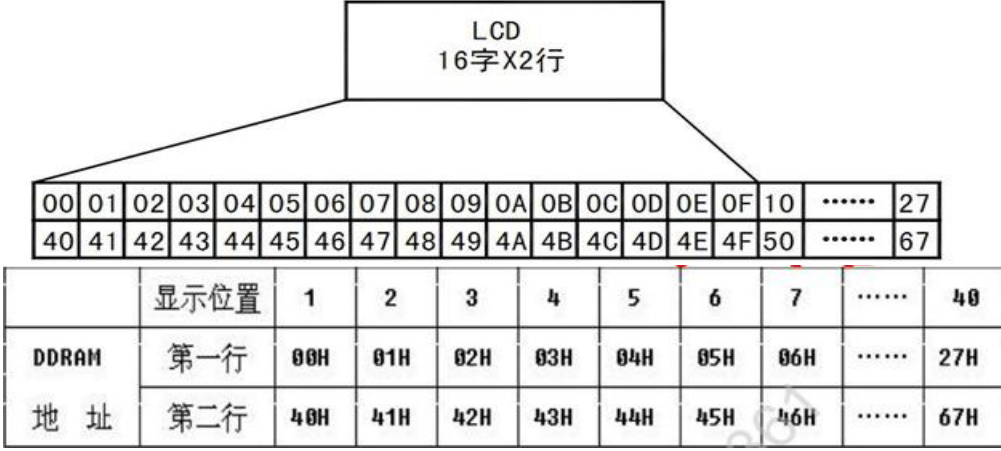


图 3 lcd1602 液晶显示屏

在 LCD1602 内部含有 80 个字节的 DDRAM，它是用来寄存显示字符的。其地址 和屏幕的对应关系如下表



从上图可知，不是所有的地址都可以直接用来显示字符数据，只有第一行中的 00-0F，第二行中的 40-4F 才能显示，其他地址只能用于存储。要显示字符时 要先输入显示字符地址，也就是告诉模块在哪里显示字符，例如第二行第一个字 符的地址是 40H，那么是否直接写入 40H 就可以将光标定位在第二行第一个字 符的位置呢？这样不行，因为写入显示地址时要求最高位 D7 恒定为高电平 1 所以 实际写入的数据应该是 01000000B(40H)。

(1) 清屏指令

指令功能	指令编码										执行时间 /us
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
进入模式设置	0	0	0	0	0	0	0	1	I/D	S	40

功能：
清除液晶显示器，即将 DDRAM 的内容全部填入“空白”的 ASCII 码 20H；
光标归位，即将光标撤回液晶显示屏的左上方；
将地址计数器(AC)的值设为 0。

(2) 模式设置指令

指令功能	指令编码										执行时间 /ms
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
清屏	0	0	0	0	0	0	0	0	0	1	1.64

功能：
设定每次写入 1 位数据后光标的移位方向，并且设定每次写入的一个字是否

移动

I/D: 0=写入新数据后光标左移

1=写入新数据后光标右

S: 0=写入新数据后显示屏不移动

1=写入新数据后显示屏整体右移 1

个字符

(3) 显示开关控制指令

指令功能	指令编码										执行时间/ μ s
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
显示开关控制	0	0	0	0	0	0	1	D	C	B	40

功能:

控制显示器开/关、光标显示/关闭以及光标是否闪烁。

D: 0=显示功能关

1=显示功能开

C: 0=无光标

1=有光标

B: 0=光标闪烁

1=光标不闪烁

(4) 功能设定指令

指令功能	指令编码										执行时间/ μ s
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
功能设定	0	0	0	0	1	DL	N	F	X	X	40

功能:

设定数据总线位数、显示的行数及字型。

DL: 0=数据总线为 4 位

1=数据总线为 8 位

N: 0=显示 1 行

1=显示 2 行

F: 0=5 $\times$ 7 点阵/每字符

1=5 $\times$ 10 点阵/每字符

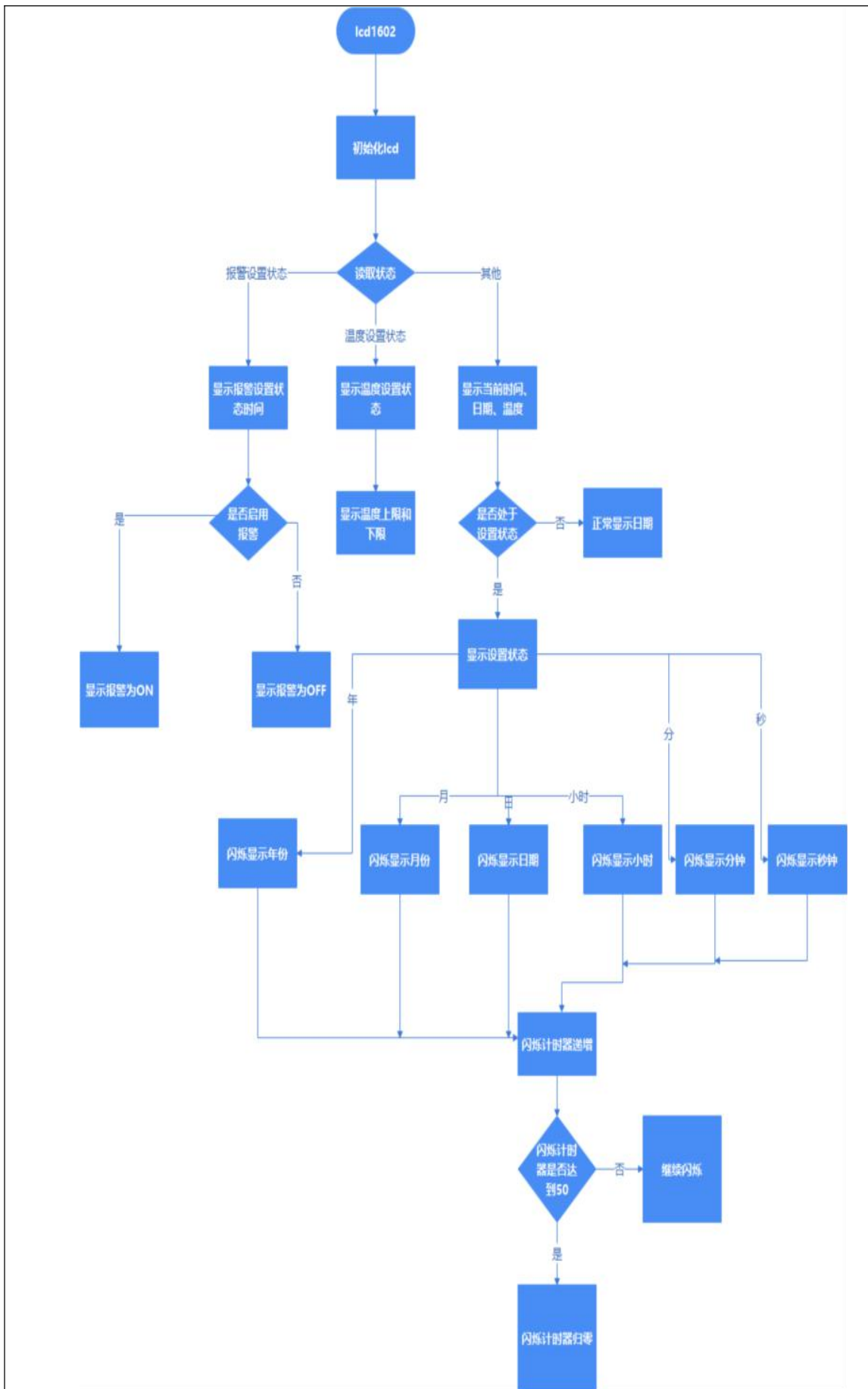


图 4 显示模块流程图设计

3.2 键盘功能模块：

- 采用 4×4 矩阵键盘作为输入设备，通过单片机的 I/O 口进行扫描。
- 键盘功能包括时间设定、闹铃设定、温度设定等。
- 按键响应速度快，操作简单、方便。

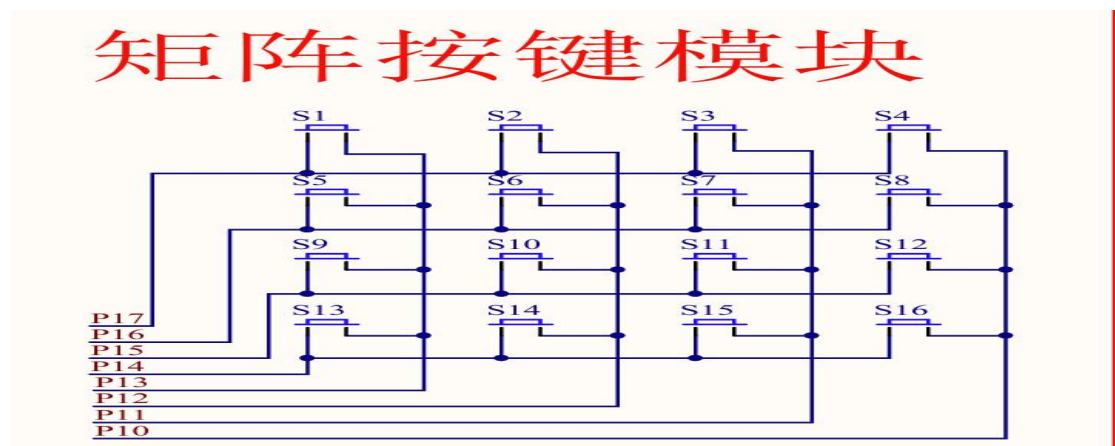


图 5 矩阵键盘

从上图中可以看出， 4×4 矩阵按键引出的 8 根控制线直接连接到 51 单片机的 P1 口上。电路中的 P17 连接矩阵键盘的第 1 行，P13 连接矩阵键盘第 1 列。

在检测时需编程通过单片机 I/O 口送出低电平。检测方法有多种，最常用的是 行列扫描和线翻转法。

行列扫描法：

原理：

先从 P1 口的高四位（四个行）输出高电平，低四位（四个列）输出低电平，假设有按键按下，从 P1 口的高四位读取键盘状态。判断高四位的四行哪一行变成了低电平，就知道是第几行，再从 P1 口的低四位（四个列）输出高电平，高四位（四个行）输出低电平，从 P1 口的低四位读取键盘状态。判断低四位的四列哪一行变成了低电平，就知道是第几列，将两次读取结果组合起来就可以得到当前按键的特征编码。使用上述方法我们得到 16 个键的特征编码。

线翻转法：就是使所有行线为低电平时，检测所有列线是否有低电平，如果有，就记录列线值；然后再翻转，使所有列线都为低电平，检测所有行线的值，由于有按键按下，行线的值也会有变化，记录行线的值。从而就可以检测到全部按键。

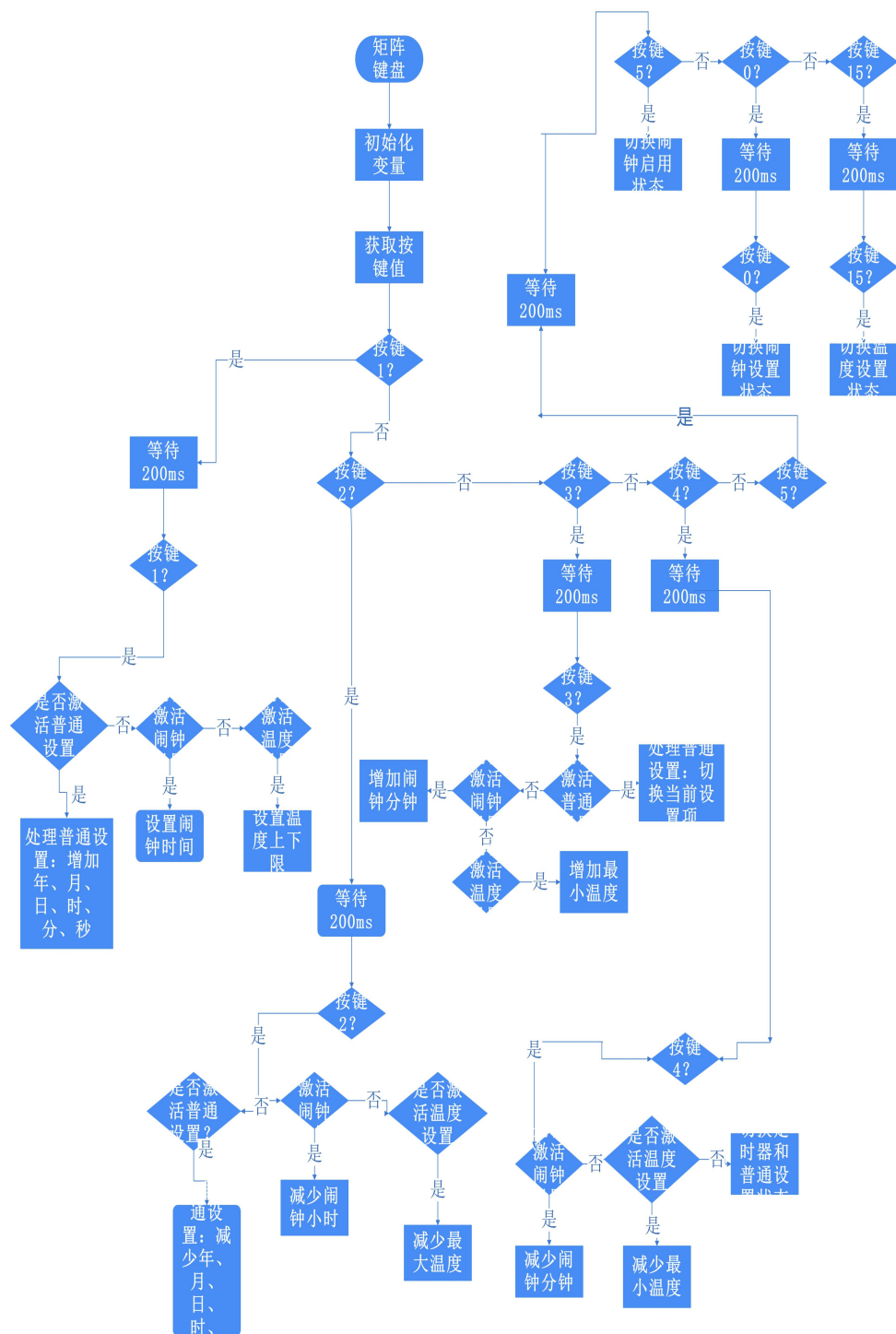


图 6 按键模块的流程图设计

日期：6 月 19 日

实习（实训）内容：

3.3 闹铃、提醒功能模块：

- 采用单片机的定时/计数器实现闹铃和提醒功能。
- 用户可通过键盘设定闹铃时间、提醒时间和温度上下限，当时间到达设定值时，系统发出声报警。
- 闹铃和提醒功能可根据用户需求进行开启或关闭。



图 7 蜂鸣器

蜂鸣器按驱动方式可分为有源蜂鸣器（内含驱动线路）和无源蜂鸣器（外部驱动）。这里的“源”指的是激励源。

无源蜂鸣器内部没有激励源，只有给它一定频率的方波信号，才能让蜂鸣器的振动装置起振，从而实现发声，同时，输入的方波频率不同，发出的声音也不同（所以无源蜂鸣器可以模拟曲调实现音乐效果）。

有源蜂鸣器则不需要外部的激励源，只需要接入直流电源，即可自动发出声音（声音频率相对固定）。

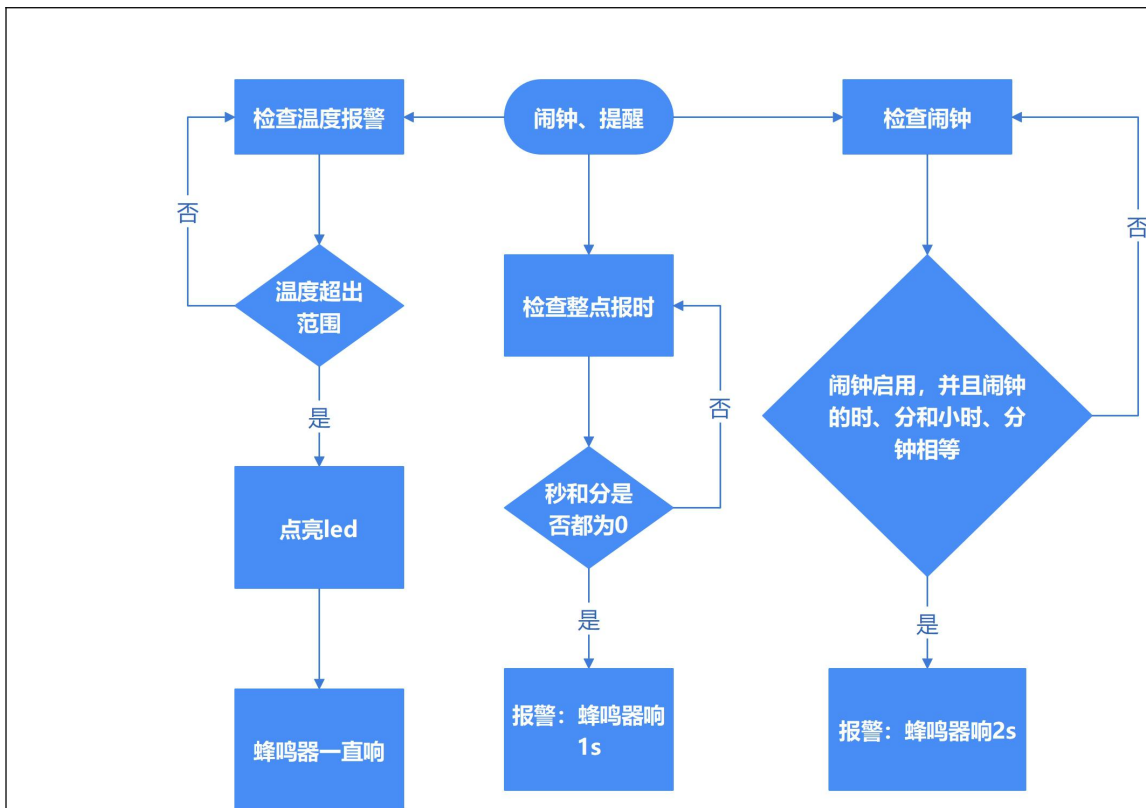


图 8 闹钟、提醒模块流程图设计

3.4 温度测量功能模块：

- 采用 DS18B20 数字温度传感器作为温度测量器件，通过单片机的 I/O 口进行通信。
- 温度传感器精度高、性能稳定，可实时采集周围环境温度。
- 温度测量范围为 $-55^{\circ}\text{C} \sim +125^{\circ}\text{C}$ ，测量精度为 $\pm 0.5^{\circ}\text{C}$ 。

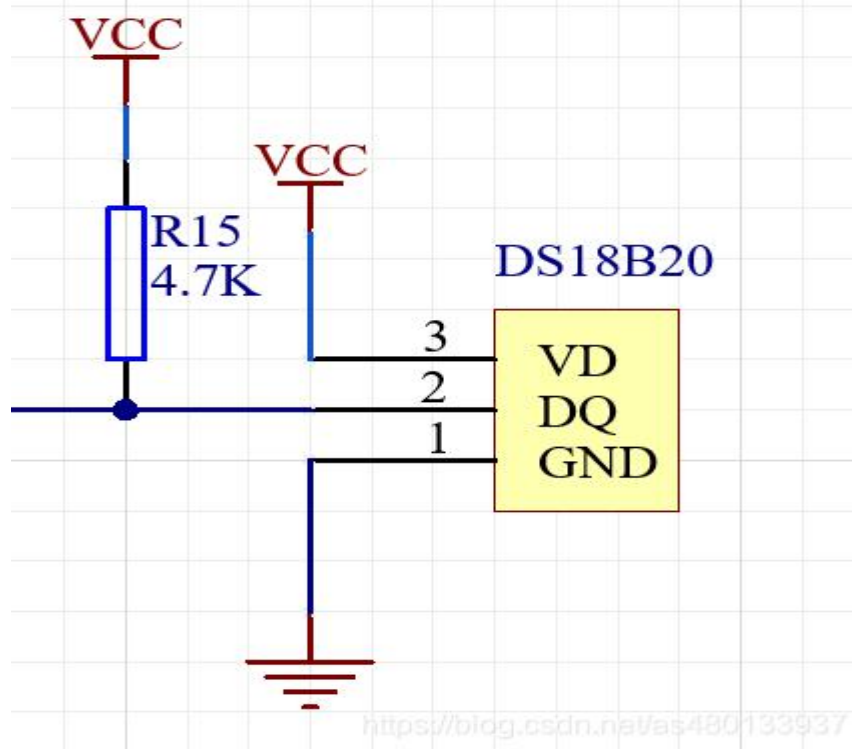
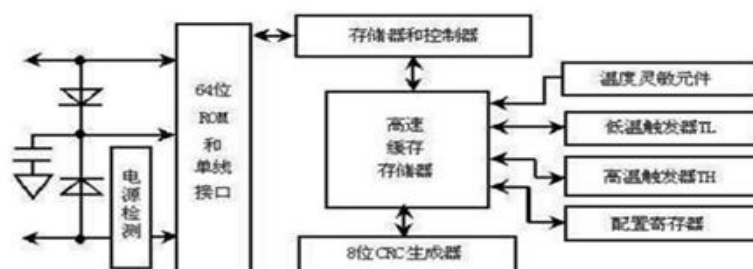


图 9 DS18B20

单个 DS18B20 接线方式：VDD 接到电源，DQ 接单片机的引脚，同时外加上拉电阻，GND 接地

DS18B20 是单线通信，即接收和发送都是这个通信脚进行的。其接收数据时为高电阻输入，其发送数据时是开漏输出，本身不具有输出高电平的能力，即输出 0 时通过 MOS 下拉为低电平，而输出 1 时，则为高阻，需要外接上拉电阻将其拉为高电平。因此，需要外接上拉电阻，否则无法输出 1。

DS18B20 内部结构如下图所示：



ROM 中的 64 位序列号是出厂前被光刻好的，它可以看作是 DS18B20 的地址序列号。64 位光刻 ROM 的排列是：开始 8 位 (28H) 是产品类型标号，接

着的 48 位是该 DS18B20 自身的序列号，最后 8 位是前面 56 位的循环冗余校验码。光刻 ROM 的作用是使每一个 DS18B20 都各不相同，这样就可以实现一根总线上挂接多个 DS18B20 的目的。

DS18B20 工作步骤：

DS18B20 的工作步骤可以分为三步：

1. 初始化 DS18B20
2. 执行 ROM 指令
3. 执行 DS18B20 功能指令

其中第二步执行 ROM 指令，也就是访问每个 DS18B20，搜索 64 位序列号，读取匹配的序列号值，然后匹配对应的 DS18B20，如果我们仅仅使用单个 DS18B20，可以直接跳过 ROM 指令。而跳过 ROM 指令的字节是 0xCC。

DS18B20 写入的功能命令：

ROM 指令：

采用多个 DS18B20 时，需要写 ROM 指令来控制总线上的某个 DS18B20

如果是单个 DS18B20，直接写跳过 ROM 指令 0xCC 即可

指令名称	指令代码	指令功能
读 ROM	33H	读 DS18B20ROM 中的编码（即读 64 位地址）
ROM 匹配（符合 ROM）	55H	发出此命令之后，接着发出 64 位 ROM 编码，访问单总线上与编码相对应 DS18B20 使之作出响应，为下一步对该 DS18B20 的读写作准备
搜索 ROM	0F0H	用于确定挂接在同一总线上 DS18B20 的个数和识别 64 位 ROM 地址，为操作各器件作好准备
跳过 ROM	0CCH	忽略 64 位 ROM 地址，直接向 DS18B20 发温度变换命令，适用于单片机工作
警报搜索	0ECH	该指令执行后，只有温度超过设定值上限或下限的片子才做出响应 https://blog.csdn.net/ag480422933

RAM 指令，DS18B20 的一些功能指令

常用的是：

温度转换 0x44

开启温度读取转换，读取好的温度会存储在高速暂存器的第 0 个和第一个字节中

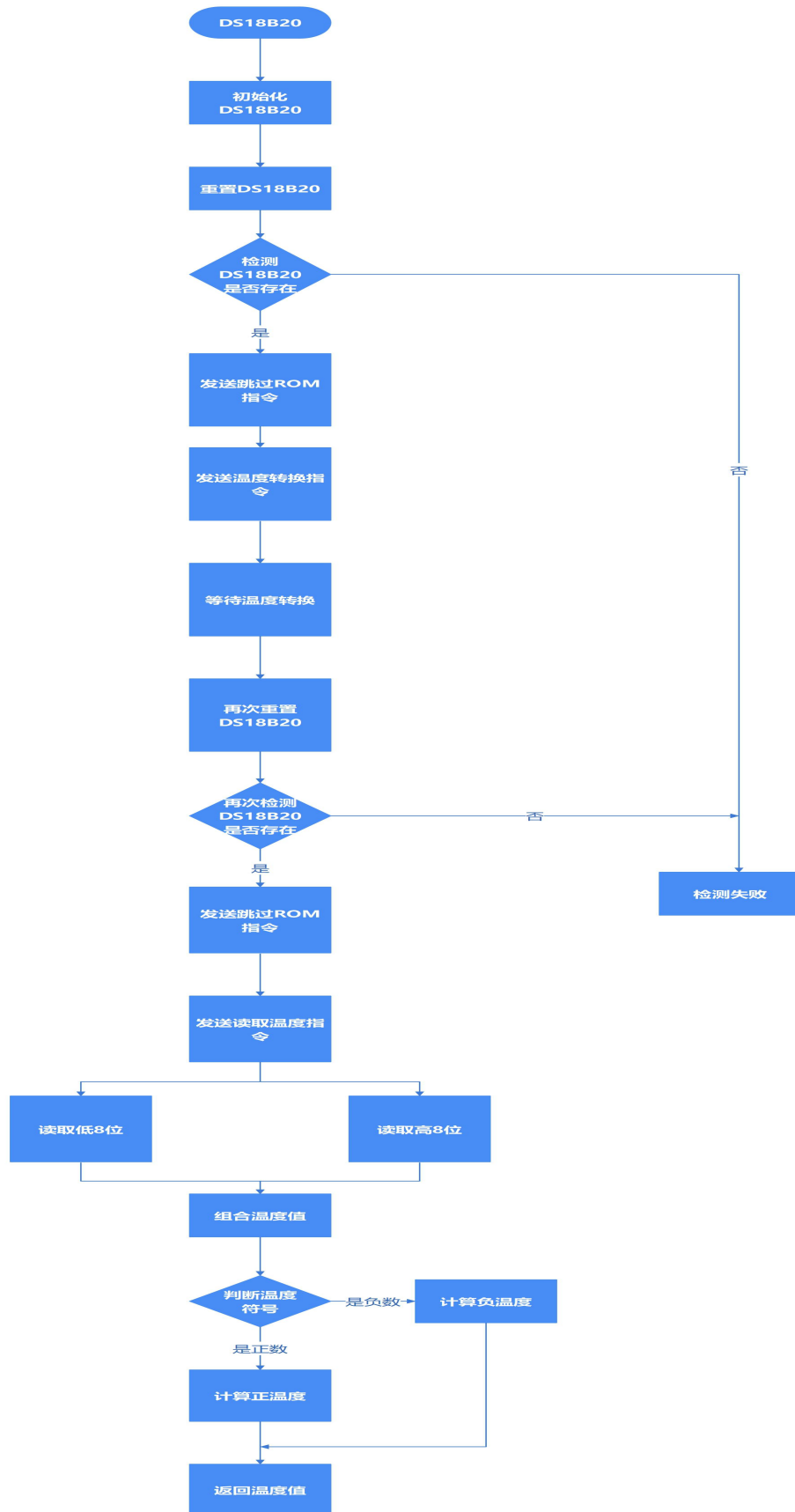


图 10 DS18B20 温度传感器流程图设计

日期：6 月 20 日

实习（实训）内容：

3.5 系统主控电路原理图设计，系统外围电路原理图设计

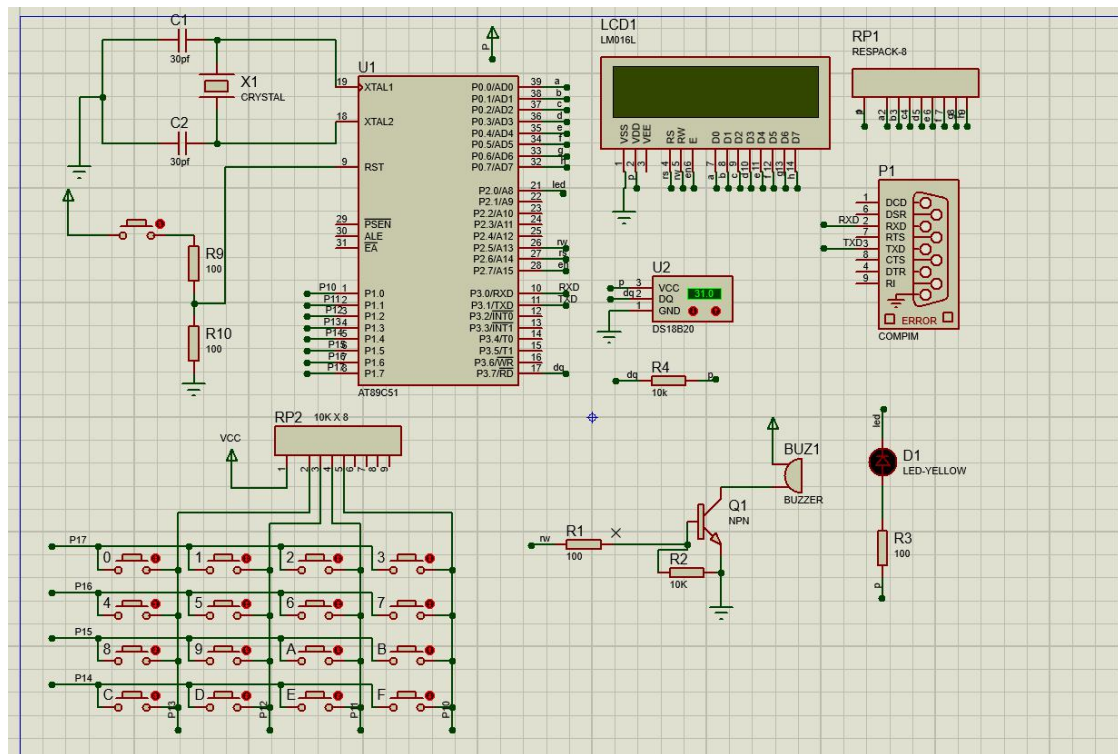


图 11 电路原理图绘制

日期：6 月 21 日

实习（实训）内容：

四、程序设计与调试

4.1 LCD 显示程序编写、调试

```
#include "lcd1602.h"
```

```
u16 y, x, week;
```

```
// 定义全局变量 y, x,
```

week, 用于计算星期

```
u8 show_date[16] = {""};
```

```
// 显示日期的字符数组
```

```
u8 show_time[16] = {" "};
```

```
// 显示时间的字符数组，默认
```


空格

```
u8 blink_counter          = 0;                // 闪烁计数器
u8 show_temp[16]          = {" "};           // 显示温度的字符数组
u8 show_temp_upper[16] = {"upper:"}; // 显示温度上限的字符数组
u8 show_temp_lower[16] = {"lower:"};
u8 days_in_month[12]      = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30,
31}; // 每月天数数组
u8 days_of_week[7][4] = {"Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"};
// 星期数组
bit alarm_enabled = 0; // 报警使能位
int max = 35; // 温度上限
int min = 20; // 温度下限
bit is_leap_year(u16 year)
{
return ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)); // 判
断是否闰年
}
int second = 56; // 初始化秒
int minute = 58; // 初始化分
int hour    = 9;  // 初始化时
u16 year = 2024; // 初始化年
u8 month = 6;    // 初始化月
u8 day    = 19;  // 初始化日
int alarm_hour    = 9; // 报警小时
int alarm_minute = 59; // 报警分钟
u8 saved_alarm_hour    = 9; // 保存的报警小时
u8 saved_alarm_minute = 59; // 保存的报警分钟
```

```

u8 saved_max = 35;

u8 saved_min = 20;

u16 temp_value;          // 温度值

u8 current_week_index; // 当前星期索引

u8 get_week_of_date(u16 year, u8 month, u8 day)
{
    if (month < 3) {
        month += 12; // 若月份小于 3，年份减 1，月份加 12
        year -= 1;
    }

    y = year % 100; // 计算 y
    x = year / 100; // 计算 x

    week = (day + ((13 * (month + 1)) / 5) + y + (y / 4) + (x / 4) - 2 * x) %
    7; // Zeller 公式计算星期

    // Zeller 公式得到的星期索引，转换为 0-6 对应周日到周六

    if (week == 0) {
        week = 7;
    }

    return week - 1;
}

u8 get_days_in_month(u8 month, u16 year)
{
    if (month == 2 && is_leap_year(year)) {
        return 29; // 若是闰年且是 2 月，返回 29 天
    } else {
        return days_in_month[month - 1]; // 否则返回对应月份的天数
    }
}

```

```

}

void lcd1602_write_cmd(u8 cmd)
{
    RS = 0;    // RS 置 0, 表示写命令
    RW = 0;    // RW 置 0, 表示写模式
    EN = 1;    // EN 置 1, 使能 LCD
    P0 = cmd;  // 将命令写入 P0 口
    delay(1);  // 延时 1 毫秒
    EN = 0;    // 关闭 LCD 使能
}

void lcd1602_write_dat(u8 dat)
{
    RS = 1;    // RS 置 1, 表示写数据
    RW = 0;    // RW 置 0, 表示写模式
    EN = 1;    // EN 置 1, 使能 LCD
    P0 = dat;  // 将数据写入 P0 口
    delay(1);  // 延时 1 毫秒
    EN = 0;    // 关闭 LCD 使能
}

void lcd1602_init()
{
    lcd1602_write_cmd(0x38); // 初始化命令, 8 位数据接口, 两行显示, 5x7 点
    阵
    lcd1602_write_cmd(0x0c); // 开启显示, 光标不闪烁
    lcd1602_write_cmd(0x06); // 光标右移, 整体不移动
    lcd1602_write_cmd(0x01); // 清屏, 光标返回初始位置
}

```

```

void lcd1602_display_time()
{
    int i;

    temp_value = ds18b20_read_temperature() * 10; // 读取并计算温度值
    if (alarm_setting_active) { // 如果处于报警
        设置状态
        lcd1602_write_cmd(0x80); // 设置显示位置为第一行第一个字符
        show_time[0] = 't'; // 显示字符串"time"
        show_time[1] = 'i';
        show_time[2] = 'm';
        show_time[3] = 'e';
        show_time[4] = ' ';
        show_time[5] = ':';
        show_time[6] = ' ';
        show_time[7] = ' ';
        show_time[8] = hour / 10 + '0'; // 小时十位数
        show_time[9] = hour % 10 + '0'; // 小时个位数
        show_time[10] = ':';
        show_time[11] = minute / 10 + '0'; // 分钟十位数
        show_time[12] = minute % 10 + '0'; // 分钟个位数
        show_time[13] = ':';
        show_time[14] = second / 10 + '0'; // 秒钟十位数
        show_time[15] = second % 10 + '0'; // 秒钟个位数
        for (i = 0; i < 16; i++) {
            lcd1602_write_dat(show_time[i]); // 循环写入显示时间数据
        }

        lcd1602_write_cmd(0xc0); // 设置显示位置为第二行第一个字符
    }
}

```

```

show_time[0]    = 'a';          // 显示字符串"alarm"
show_time[1]    = 'l';
show_time[2]    = 'a';
show_time[3]    = 'r';
show_time[4]    = 'm';
show_time[5]    = ':';
show_time[6]    = alarm_hour / 10 + '0'; // 报警小时十位数
show_time[7]    = alarm_hour % 10 + '0'; // 报警小时个位数
show_time[8]    = ':';
show_time[9]    = alarm_minute / 10 + '0'; // 报警分钟十位数
show_time[10]   = alarm_minute % 10 + '0'; // 报警分钟个位数
show_time[11]   = ' ';
show_time[12]   = ' ';
show_time[13]   = ' ';
show_time[14]   = ' ';
show_time[15]   = ' ';
if (alarm_enabled) {
show_time[13]   = '0'; // 若报警使能，显示"ON"
show_time[14]   = 'N';
} else {
show_time[13]   = '0'; // 若报警未使能，显示"OFF"
show_time[14]   = 'F';
show_time[15]   = 'F';
}
for (i = 0; i < 16; i++) {
lcd1602_write_dat(show_time[i]); // 循环写入显示报警数据
}

```

```

} else if (temp_setting_active) {
    lcd1602_write_cmd(0x80); // 设置显示位置为第一行第一个字符
    show_temp_upper[6] = max / 10 + '0';
    show_temp_upper[7] = max % 10 + '0';
    show_temp_upper[8] = 0xdf;
    show_temp_upper[9] = 'c';
    for (i = 0; i < 12; i++) {
        lcd1602_write_dat(show_temp_upper[i]); // 循环写入显示报警数据
    }
    lcd1602_write_cmd(0x8a);
    for (i = 0; i < 6; i++) {
        lcd1602_write_dat(' '); // 循环写入显示报警数据
    }
    lcd1602_write_cmd(0xc0);
    show_temp_lower[6] = min / 10 + '0';
    show_temp_lower[7] = min % 10 + '0';
    show_temp_lower[8] = 0xdf;
    show_temp_lower[9] = 'c';
    for (i = 0; i < 12; i++) {
        lcd1602_write_dat(show_temp_lower[i]); // 循环写入显示报警数据
    }
    lcd1602_write_cmd(0xca);
    for (i = 0; i < 6; i++) {
        lcd1602_write_dat(' '); // 循环写入显示报警数据
    }
} else { // 非报警设置状态
    show_date[0] = year / 1000 + '0'; // 年份千位数

```

```

show_date[1]    = year % 1000 / 100 + '0'; // 年份百位数
show_date[2]    = year % 100 / 10 + '0';    // 年份十位数
show_date[3]    = year % 10 + '0';          // 年份个位数
show_date[4]    = '-';
show_date[5]    = month / 10 + '0'; // 月份十位数
show_date[6]    = month % 10 + '0'; // 月份个位数
show_date[7]    = '-';
show_date[8]    = day / 10 + '0'; // 日份十位数
show_date[9]    = day % 10 + '0'; // 日份个位数
show_date[10]   = ' ';
show_date[11]   = ' ';
show_date[12]   = ' ';
show_time[0]    = hour / 10 + '0'; // 小时十位数
show_time[1]    = hour % 10 + '0'; // 小时个位数
show_time[2]    = ':';
show_time[3]    = minute / 10 + '0'; // 分钟十位数
show_time[4]    = minute % 10 + '0'; // 分钟个位数
show_time[5]    = ':';
show_time[6]    = second / 10 + '0'; // 秒钟十位数
show_time[7]    = second % 10 + '0'; // 秒钟个位数
show_time[8]    = ' ';
show_time[9]    = ' ';
lcd1602_write_cmd(0x80); // 设置显示位置为第一行第一个字符
for (i = 0; i < 13; i++) {
    lcd1602_write_dat(show_date[i]); // 循环写入显示日期数据
}
current_week_index = get_week_of_date(year, month, day); // 获取当前日

```

期的星期索引

```
lcd1602_write_cmd(0x8d); // 设置显示位置为第一行第 14 个字符
for (i = 0; i < 3; i++) {
    lcd1602_write_dat(days_of_week[current_week_index][i]); // 循环写入当前星期数据
}

lcd1602_write_cmd(0xc0); // 设置显示位置为第二行第一个字符
for (i = 0; i < 10; i++) {
    lcd1602_write_dat(show_time[i]); // 循环写入显示时间数据
}

lcd1602_write_cmd(0xca); // 设置显示位置为第二行第 11 个字符
show_temp[0] = temp_value / 100 + '0'; // 温度百位数
delay(1);
show_temp[1] = (temp_value / 10) % 10 + '0'; // 温度十位数
delay(1);
show_temp[2] = '.';
delay(1);
show_temp[3] = temp_value % 10 + '0'; // 温度个位数
delay(1);
show_temp[4] = 0xdf; // 温度单位° C
delay(1);
show_temp[5] = 'c';
for (i = 0; i < 6; i++) {
    lcd1602_write_dat(show_temp[i]); // 循环写入显示温度数据
}

if (setting_active) { // 如果处于设置状态
    switch (current_setting) {
```



```

case 0:                                     // 设置年份
if (blink_counter < 50) {                  // 闪烁显示
lcd1602_write_cmd(0x80); // 设置显示位置为第一行第一个字符
lcd1602_write_dat(' '); // 显示空格
lcd1602_write_dat(' ');
lcd1602_write_dat(' ');
lcd1602_write_dat(' ');
} else {
lcd1602_write_cmd(0x80); // 设置显示位置为第一行第一个字符
for (i = 0; i < 4; i++) {
lcd1602_write_dat(show_date[i]); // 循环写入显示日期数据
}
}
break;

case 1:                                     // 设置月份
if (blink_counter < 50) {                  // 闪烁显示
lcd1602_write_cmd(0x85); // 设置显示位置为第一行第 6 个字符
lcd1602_write_dat(' '); // 显示空格
lcd1602_write_dat(' ');
} else {
lcd1602_write_cmd(0x85); // 设置显示位置为第一行第 6 个字符
for (i = 5; i < 7; i++) {
lcd1602_write_dat(show_date[i]); // 循环写入显示日期数据
}
}
break;

case 2:                                     // 设置日期

```

```

if (blink_counter < 50) {          // 闪烁显示
lcd1602_write_cmd(0x88); // 设置显示位置为第一行第 9 个字符
lcd1602_write_dat(' '); // 显示空格
lcd1602_write_dat(' ');
} else {
lcd1602_write_cmd(0x88); // 设置显示位置为第一行第 9 个字符
for (i = 8; i < 10; i++) {
lcd1602_write_dat(show_date[i]); // 循环写入显示日期数据
}
}
break;
case 3:                                // 设置小时
if (blink_counter < 50) {          // 闪烁显示
lcd1602_write_cmd(0xc0); // 设置显示位置为第二行第一个字符
lcd1602_write_dat(' '); // 显示空格
lcd1602_write_dat(' ');
} else {
lcd1602_write_cmd(0xc0); // 设置显示位置为第二行第一个字符
for (i = 0; i < 2; i++) {
lcd1602_write_dat(show_time[i]); // 循环写入显示时间数据
}
}
break;
case 4:                                // 设置分钟
if (blink_counter < 50) {          // 闪烁显示
lcd1602_write_cmd(0xc3); // 设置显示位置为第二行第 4 个字符
lcd1602_write_dat(' '); // 显示空格

```

```

lcd1602_write_dat(' ');
} else {
lcd1602_write_cmd(0xc3); // 设置显示位置为第二行第 4 个字符
for (i = 3; i < 5; i++) {
lcd1602_write_dat(show_time[i]); // 循环写入显示时间数据
}
}
break;
case 5: // 设置秒钟
if (blink_counter < 50) { // 闪烁显示
lcd1602_write_cmd(0xc6); // 设置显示位置为第二行第 7 个字符
lcd1602_write_dat(' '); // 显示空格
lcd1602_write_dat(' ');
} else {
lcd1602_write_cmd(0xc6); // 设置显示位置为第二行第 7 个字符
for (i = 6; i < 8; i++) {
lcd1602_write_dat(show_time[i]); // 循环写入显示时间数据
}
}
break;
}
}

blink_counter++; // 闪烁计数器递增
if (blink_counter >= 50) { // 闪烁计数器归零
blink_counter = 0;
}
}
}

```

}

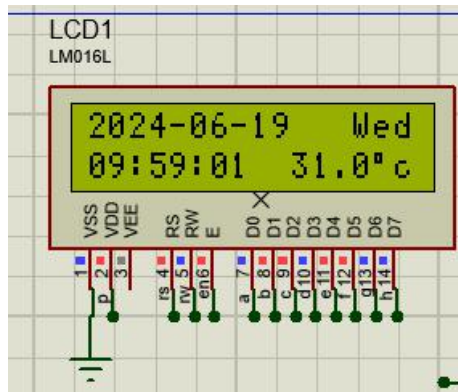


图 12 lcd1602 液晶显示调试

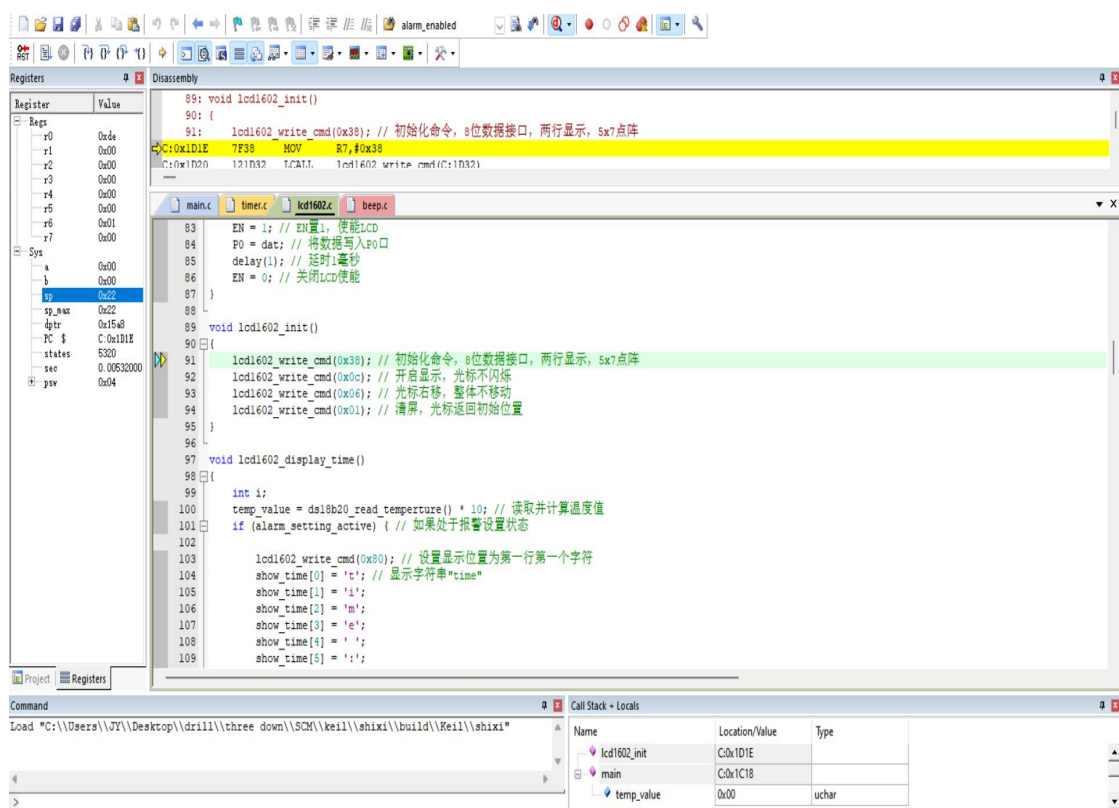


图 13 lcd1602 代码调试

4.2 DS18B20 模块程序设计、调试

```
#include "ds18b20.h"
```

```
void Ds18b20Reset() // 重置 DS18B20 传感器
```

```
{
```

```

// 将 DS18B20 引脚拉低一段时间然后拉高复位
DQ = 0;
delay_10us(75);
DQ = 1;
delay_10us(2);
}

u8 CheckDs18B20() // 检查 DS18B20 传感器是否响应
{
    u8 time_temp = 0;
    // 检测 DS18B20 设备是否存在
    while (DQ && time_temp < 20) {
        time_temp++;
        delay_10us(1);
    }
    if (time_temp >= 20)
        return 1; // 超时返回 1, 表示检测失败
    else
        time_temp = 0;
    while ((!DQ) && time_temp < 20) {
        time_temp++;
        delay_10us(1);
    }
    if (time_temp >= 20)
        return 1; // 超时返回 1, 表示检测失败
    return 0; // 返回 0, 表示检测成功
}

u8 Ds18B20ReadsOneBit() // 从 DS18B20 读取一个位

```

```

{
u8 dat = 0;
// 读取 DS18B20 的一位数据
DQ = 0;
DQ = 1;
if (DQ)
dat = 1; // 如果引脚为高电平，dat 为 1
else
dat = 0; // 如果引脚为低电平，dat 为 0
delay_10us(5); // 延时 5 * 10 微秒
return dat; // 返回读取的数据位
}

u8 Ds18B20ReadsOneByte() // 从 DS18B20 读取一个字节
{
u8 i = 0;
u8 dat = 0;
u8 temp = 0;
// 读取 DS18B20 的一个字节数据
for (i = 0; i < 8; i++) {
temp = Ds18B20ReadsOneBit(); // 读取一位数据
dat = (temp << 7) | (dat >> 1); // 将数据存入 dat 变量
}
return dat; // 返回读取的字节数据
}

void Ds18B20WritesAByte(u8 dat) // 向 DS18B20 写入一个字节
{
u8 i = 0;

```

```

u8 temp = 0;
// 向 DS18B20 写入一个字节数据
for (i = 0; i < 8; i++) {
temp = dat & 0x01; // 获取 dat 的最低位
dat >>= 1;          // dat 右移一位
if (temp) {
DQ = 0;              // 如果最低位为 1，将引脚拉低
DQ = 1;              // 然后再拉高
delay_10us(6); // 延时 6 * 10 微秒
} else {
DQ = 0;              // 如果最低位为 0，先将引脚拉低
delay_10us(6); // 然后延时 6 * 10 微秒
DQ = 1;              // 最后拉高引脚
}
}
}

void StartDs18b20() // 启动 DS18B20 传感器的测温过程
{
// 启动 DS18B20 温度转换
Ds18b20Reset();
CheckDs18B20();
Ds18B20WritesAByte(0xcc); // 写入跳过 ROM 指令
Ds18B20WritesAByte(0x44); // 写入温度转换指令
}

u8 ds18b20_init()
{
// 初始化 DS18B20

```

```

Ds18b20Reset();

return CheckDs18B20(); // 返回检测结果
}

float ds18b20_read_temperture() // 于读取 DS18B20 传感器的温度值
{
float temp;          // 温度变量
u8 dath      = 0; // 高 8 位
u8 datl      = 0; // 低 8 位
ul6 value = 0; // 温度值

ds18b20_init();          // 初始化 DS18B20
StartDs18b20();          // 启动温度转换
Ds18b20Reset();          // 复位 DS18B20
CheckDs18B20();          // 检测 DS18B20
Ds18B20WritesAByte(0xcc); // 写入跳过 ROM 指令
Ds18B20WritesAByte(0xbe); // 写入读取温度指令
datl  = Ds18B20ReadsOneByte(); // 读取低 8 位
dath  = Ds18B20ReadsOneByte(); // 读取高 8 位
value = (dath << 8) + datl;      // 组合成 16 位温度
if ((value & 0xf800) == 0xf800) {
value = (~value) + 1;
temp  = value * (-0.0625);
} else {
temp = value * 0.0625;
}

return temp; // 返回温度值
}

```

日期：6 月 24 日
实习（实训）内容：

4.3 计时功能模块程序设计、调试

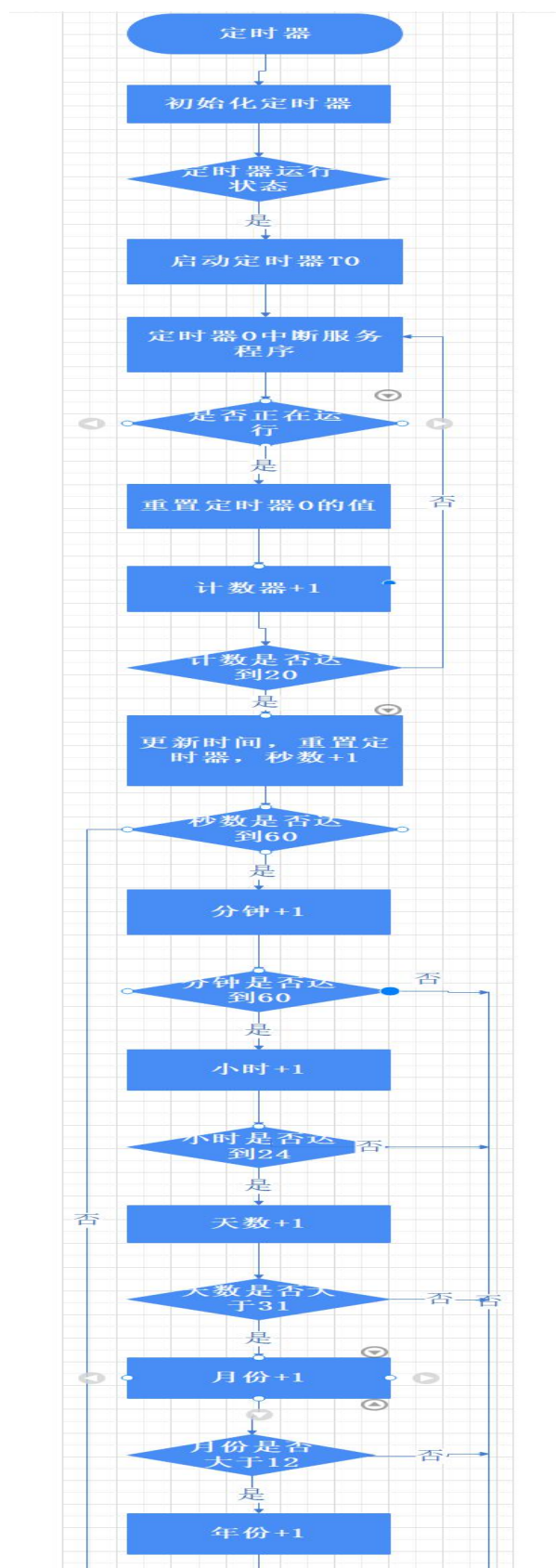


图 14 定时器流程图设计

```
#include "timer.h"

bit timer_running = 1; // 定义全局变量 timer_running，用于控制定时器运行状态，默认为 1，表示定时器正在运行

void toggle_timer()
{
    timer_running = !timer_running; // 切换定时器运行状态，取反操作
    TR0 = timer_running; // 将定时器 0 的运行状态设置为 timer_running 的值，控制定时器的启停
}

void timer0_init()
{
    TMOD = 0x01; // 设置定时器 0 为模式 1（16 位定时/计数器）
    TH0 = (65536 - 50000) / 256; // 设置定时器 0 的高 8 位初始值，使定时器周期为 50ms
    TL0 = (65536 - 50000) % 256; // 设置定时器 0 的低 8 位初始值
    ET0 = 1; // 允许定时器 0 中断
    TR0 = 1; // 启动定时器 0
    EA = 1; // 允许总中断
}

void timer0_ISR() interrupt 1
{
    if (timer_running) // 如果定时器正在运行
    {
        static unsigned char count = 0; // 静态变量，用于计数
```

```

TH0                                = (65536 - 50000) / 256; // 重
置定时器 0 的高 8 位，重新设定定时器周期为 50ms

TL0                                = (65536 - 50000) % 256; // 重
置定时器 0 的低 8 位

count++;

    // 每次定时器中断计数器加 1

if (count >=
20)                                // 当计数器
达到 20 时，表示经过 20 个 50ms，即 1 秒钟
{
count = 0; // 重置计数器
second++; // 秒数加 1
if (second >= 60) {
second = 0; // 秒数归零
minute++; // 分钟加 1
if (minute >= 60) {
minute = 0; // 分钟归零
hour++; // 小时加 1
if (hour >= 24) {
hour = 0; // 小时归零
day++; // 天数加 1
if (day > 31) {
day = 1; // 如果超过 31 天，天数重新设为 1
month++; // 月份加 1
if (month > 12) {
month = 1; // 如果超过 12 个月，月份重新设为 1
year++; // 年份加 1

```

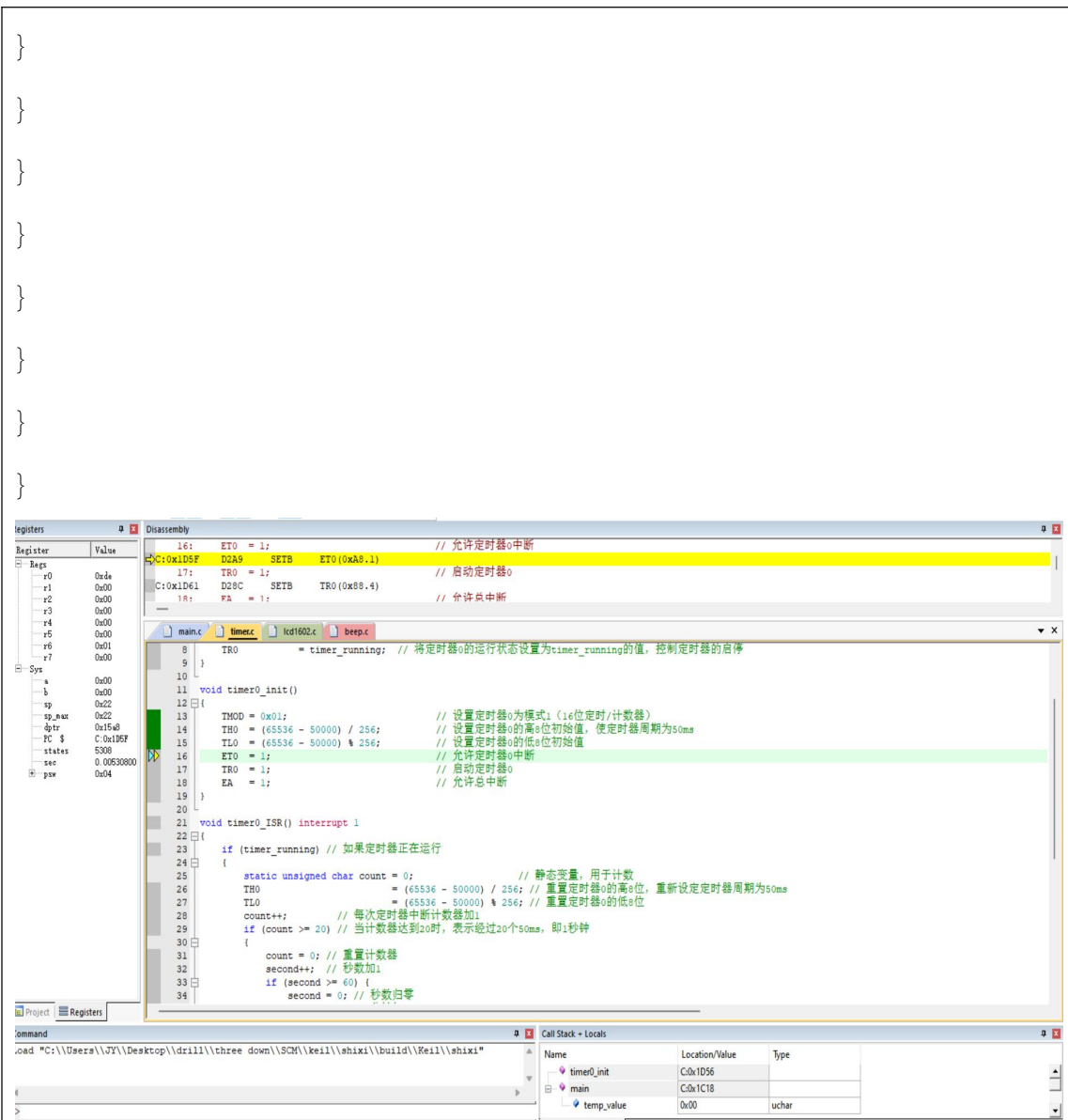


图 15 定时器代码的调试

日期：6 月 25 日

实习（实训）内容：

4.4 按键模块程序设计、调试

```
#include "key.h"
```

```
// 全局变量声明
```

```
u8 alarm_setting_active = 0; // 闹钟设置是否激活的标志
```

```
u8 temp_setting_active = 0;
```

```

u8 setting_active          = 0; // 普通设置是否激活的标志
u8 current_setting         = 0; // 当前正在设置的项目编号
u8 key; // 存储当前按键值的变量
char key_buf[] = {0x77, 0x7b, 0x7d, 0x7e,
0xb7, 0xbb, 0xbd, 0xbe,
0xd7, 0xdb, 0xdd, 0xde,
0xe7, 0xeb, 0xed, 0xee}; // 按键对应的值的数组
// 切换普通设置状态的函数
void toggle_setting()
{
    setting_active = !setting_active; // 切换普通设置激活状态
    if (setting_active) {
        current_setting = 0; // 如果激活，设置当前设置项目为 0
    }
}
// 切换闹钟设置状态的函数
void toggle_alarm_setting()
{
    alarm_setting_active = !alarm_setting_active; // 切换闹钟设置激活状态
    if (alarm_setting_active) {
        // 如果激活，恢复之前保存的闹钟时间
        alarm_hour      = saved_alarm_hour;
        alarm_minute    = saved_alarm_minute;
    } else {
        // 如果未激活，保存当前的闹钟时间
        saved_alarm_hour = alarm_hour;
        saved_alarm_minute = alarm_minute;
    }
}

```

```

}
}

void toggle_temp_setting()
{
    temp_setting_active = !temp_setting_active; //
    if (temp_setting_active) {
        // 如果激活，恢复之前保存的闹钟时间
        max = saved_max;
        min = saved_min;
    } else {
        // 如果未激活，保存当前的闹钟时间
        saved_max = max;
        saved_min = min;
    }
}

// 获取按键值的函数
char getKey()
{
    char key_scan[] = {0xbf, 0x7f, 0xef, 0xdf}; // 扫描按键时用的值
    char i = 0, j = 0;
    for (i = 0; i < 4; i++) {
        P1 = key_scan[i];           // 将当前扫描值写入 P1 端口
        if ((P1 & 0x0f) != 0x0f) // 如果低四位不全为 1
            delay(10);           // 延时 10 毫秒
        for (j = 0; j < 16; j++)
            if (key_buf[j] == P1) // 检查当前扫描值是否与预设的按键值相符
                return j;        // 返回对应的按键序号
    }
}

```

```

}

return -1; // 如果未检测到按键则返回-1

}

// 按键扫描处理函数

void key_scan()
{
    key = getKey(); // 获取当前按键值

    // 处理按键 1 的操作
    if (key == 1) {
        delay(200);
        if (key == 1) {
            if (setting_active) {
                switch (current_setting) {
                    case 0:
                        year++;
                        if (year > 2030) year = 0;
                        break;
                    case 1:
                        month++;
                        if (month > 12) {
                            month = 1;
                            year++;
                        }
                        break;
                    case 2:
                        day++;
                        if (day > get_days_in_month(month, year)) {

```

```

day = 1;
month++;
if (month > 12) {
month = 1;
}
}
break;
case 3:
hour++;
if (hour >= 24) {
hour = 0;
day++;
}
break;
case 4:
minute++;
if (minute >= 60) {
minute = 0;
hour++;
if (hour >= 24) {
hour = 0;
day++;
if (day > days_in_month[month - 1]) {
day = 1;
month++;
if (month > 12) {
month = 1;

```



```

year++;
if (year > 2030) year = 0;
}
}
}
}

break;

case 5:
second++;
if (second >= 60) {
second = 0;
minute++;
if (minute >= 60) {
minute = 0;
hour++;
if (hour >= 24) {
hour = 0;
day++;
if (day > days_in_month[month - 1]) {
day = 1;
month++;
if (month > 12) {
month = 1;
year++;
if (year > 2030) year = 0;
}
}
}
}
}

```

```

}
}
}
break;
}
} else if (alarm_setting_active) {
alarm_hour++;
if (alarm_hour > 23) alarm_hour = 0;
} else if (temp_setting_active) {
max++;
if (max > 40) max = 0;
}
}
}

// 处理按键 2 的操作，其他按键处理类似
if (key == 2) {
delay(200);
if (key == 2) {
if (setting_active) {
switch (current_setting) {
case 0:
year--;
if (year < 0) year = 2030;
break;
case 1:
month--;
if (month < 1) {

```

```
month = 12;
year--;
if (year < 0) {
year = 2030;
}
}
break;
case 2:
day--;
if (day < 1) {
day = 31;
month--;
}
break;
case 3:
hour--;
if (hour < 0) {
hour = 23;
day--;
}
break;
case 4:
minute--;
if (minute < 0) {
minute = 59;
hour--;
if (hour < 0) hour = 23;
```

```

}
break;
case 5:
second--;
if (second < 0) {
second = 59;
minute--;
if (minute < 0) {
minute = 59;
hour--;
if (hour < 0) hour = 23;
}
}
break;
}
} else if (alarm_setting_active) {
alarm_hour--;
if (alarm_hour < 0) alarm_hour = 23;
} else if (temp_setting_active) {
max--;
if (max < 0) max = 40;
}
}
}

// 处理按键 3 的操作，其他按键处理类似
if (key == 3) {
delay(200);

```

```

if (key == 3) {
    if (setting_active) {
        current_setting++;
        if (current_setting > 5) current_setting = 0;
    } else if (alarm_setting_active) {
        alarm_minute++;
        if (alarm_minute > 59) alarm_minute = 0;
    } else if (temp_setting_active) {
        min++;
        if (min > 40) min = 0;
    }
}

// 处理按键 4 的操作，其他按键处理类似
if (key == 4) {
    delay(200);
    if (key == 4) {
        if (alarm_setting_active) {
            alarm_minute--;
            if (alarm_minute < 0) alarm_minute = 59;
        } else if (temp_setting_active) {
            min--;
            if (min < 0) min = 40;
        } else {
            toggle_timer();
            toggle_setting();
        }
    }
}

```

```

}

}

// 处理按键 5 的操作，其他按键处理类似
if (key == 5) {
    delay(200);
    if (key == 5) {
        alarm_enabled = !alarm_enabled;
    }
}

// 处理按键 0 的操作，用于切换闹钟设置状态
if (key != -1) {
    if (key == 0) {
        delay(200);
        if (key == 0) {
            toggle_alarm_setting();
        }
    }
}

if (key != -1) {
    if (key == 15) {
        delay(200);
        if (key == 15) {
            toggle_temp_setting();
        }
    }
}
}

```

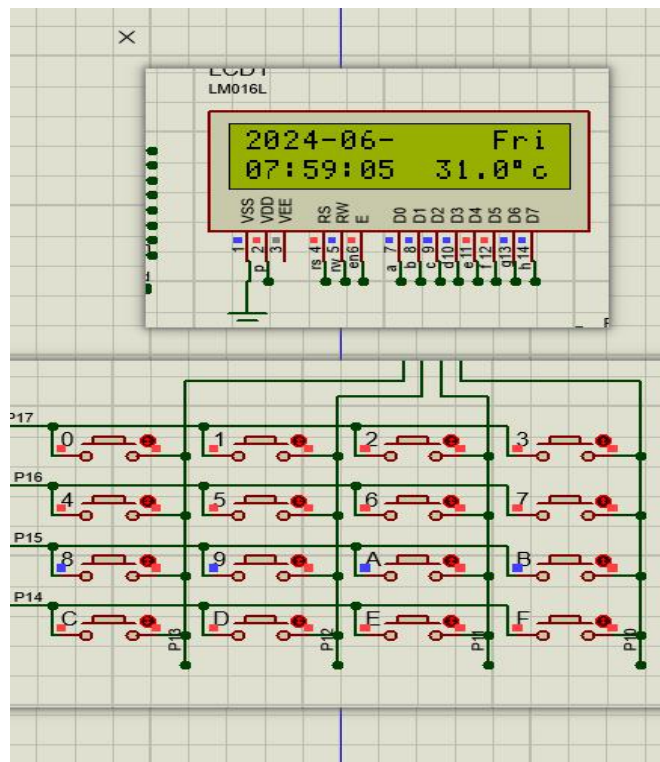


图 17 调试按键功能，日在闪烁，对日作出设置

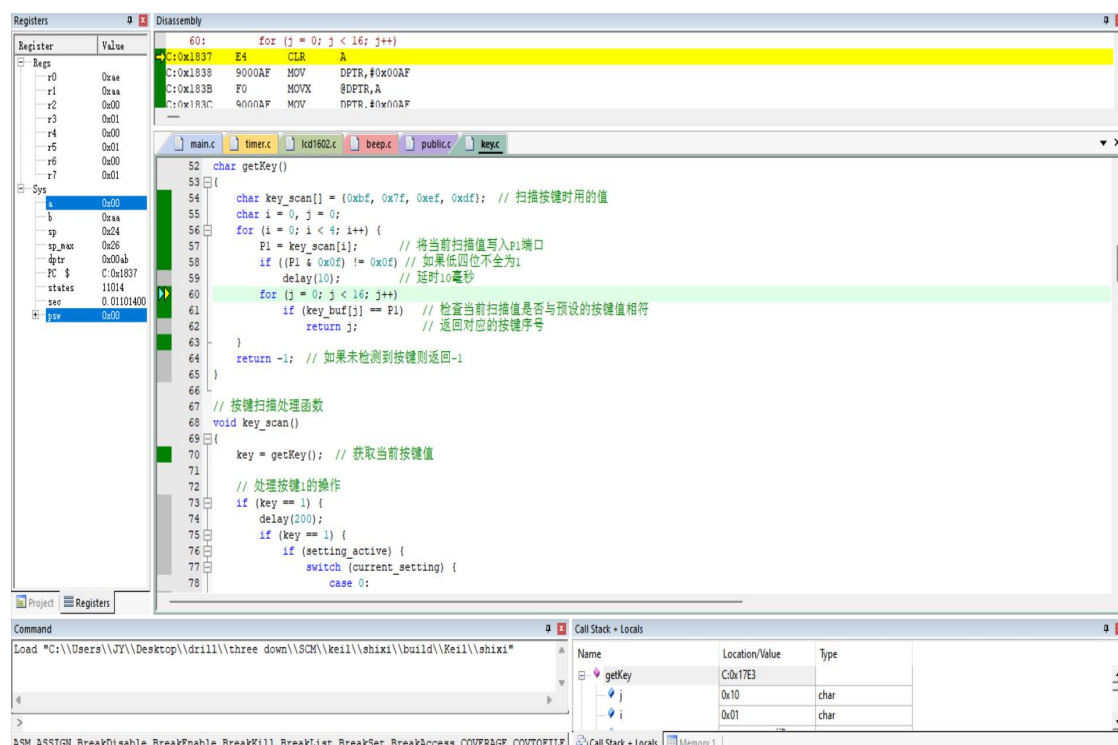


图 18 矩阵按键的代码调试

日期：6月26日

实习（实训）内容：

4.5 闹钟和整点报时模块程序设计、调试

```
#include "beep.h"

void beep_init()
{
    BEEP_PIN = 0; // 初始化蜂鸣器引脚为低电平，即关闭状态
}

void beep_on()
{
    BEEP_PIN = 1; // 打开蜂鸣器，将引脚置高电平
}

void beep_off()
{
    BEEP_PIN = 0; // 关闭蜂鸣器，将引脚置低电平
}

void HourlyReporting()
{
    // 检查是否每小时报警
    if (second == 0 && minute == 0) {
        unsigned int i;
        for (i = 0; i < 500; i++) {
            beep_on();
            delay(1);
            beep_off();
        }
    }
}
```



```

void AlarmClock()
{
// 检查是否触发设定的闹钟
if (second == 0 && minute == saved_alarm_minute && hour == saved_alarm_hour
&& alarm_enabled) {
ul6 i;
for (i = 0; i < 1000; i++) {
beep_on();
delay(1);
beep_off();
}
}
}

void TemperatureAlarm(int temperature)
{
ul6 i;
if (temperature >= saved_max || temperature <= saved_min) {
LED = 0; // 控制 LED 灯亮
for (i = 0; i < 500; i++) {
beep_on();
delay(1);
beep_off();
}
LED = 1;
delay(500);
}
}
}

```



图 19 温度报警调试，超过 34℃开始报警，led 闪



图 20 闹钟设置，状态调试

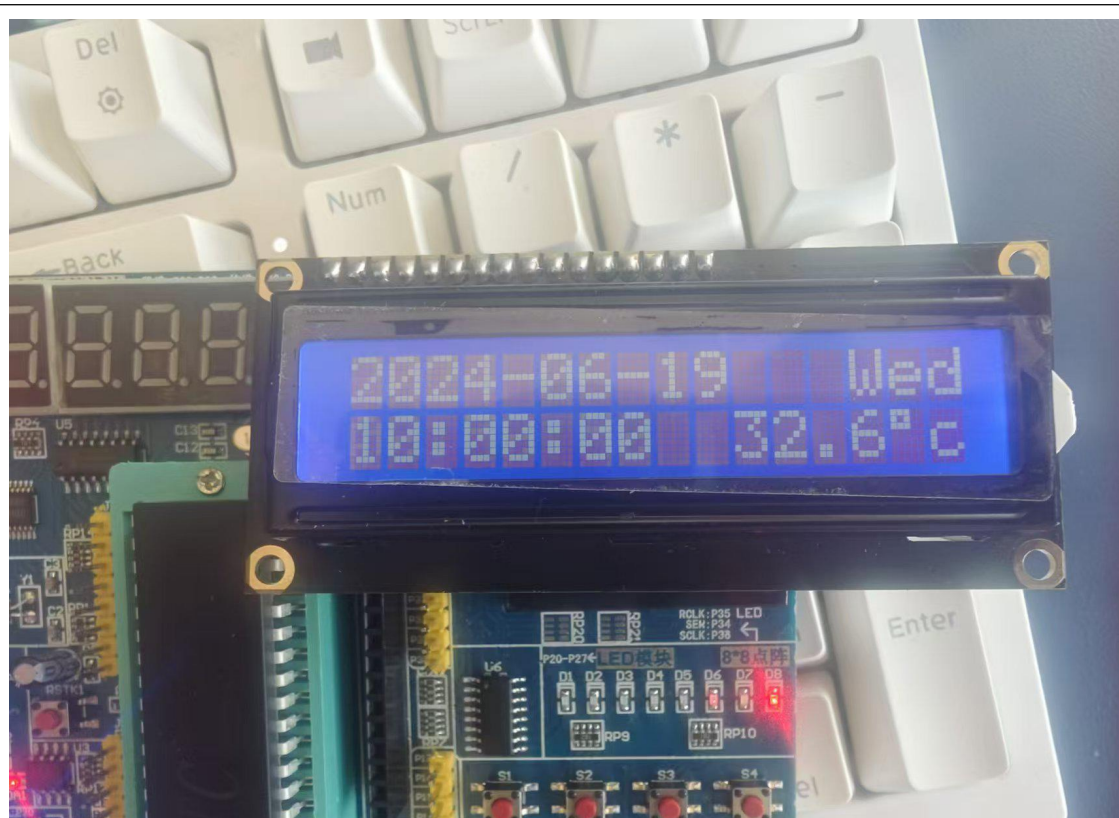


图 21 整点报时调试

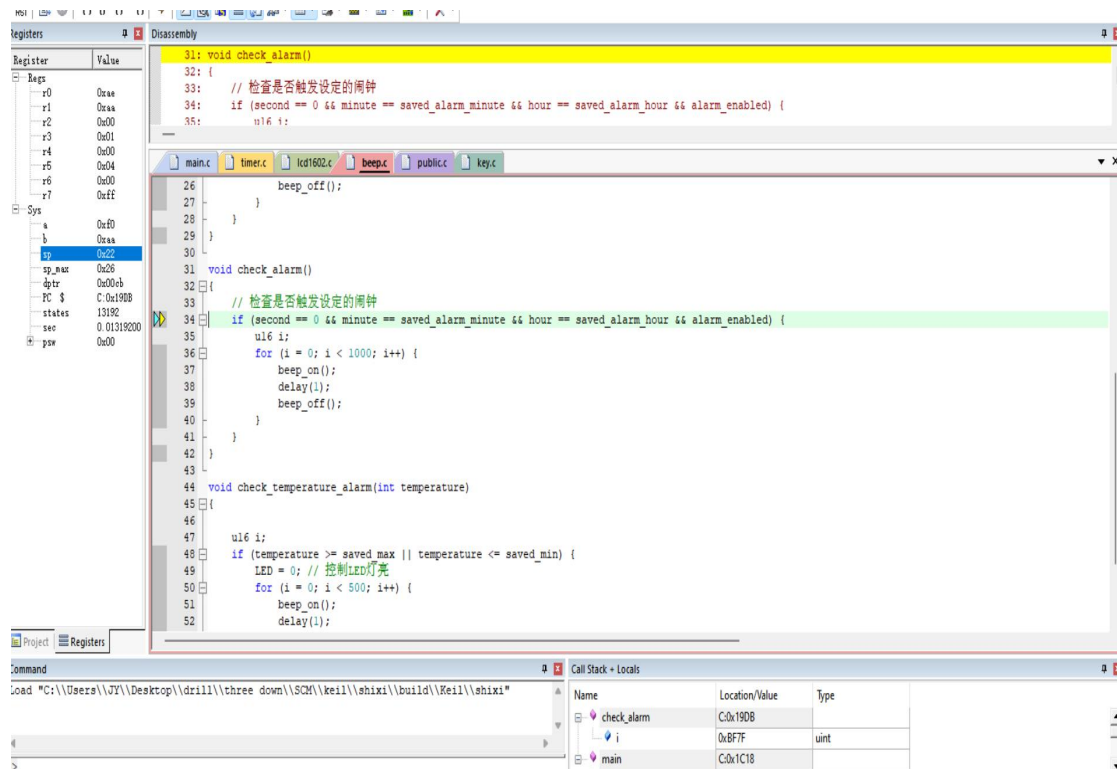


图 22 在代码中调试闹钟、整点报时等

4.6 串口模块设计、实现

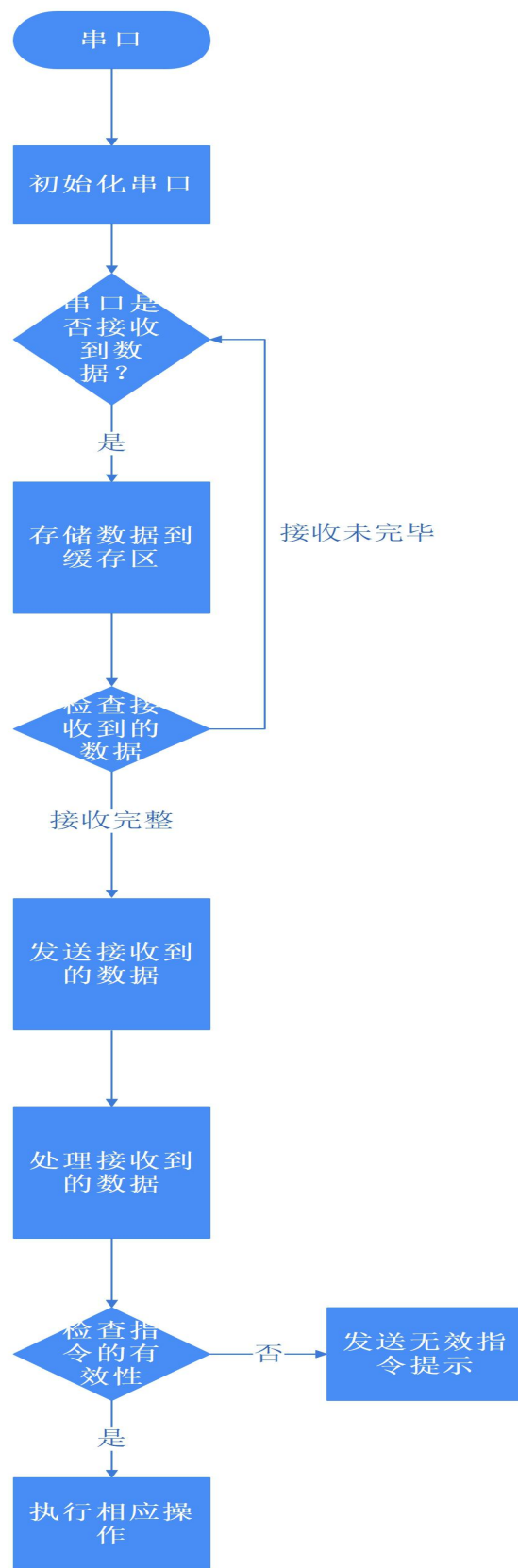


图 23 串口流程图设计

```

#include "port.h"

u8 rx_buffer[20]; // 接收缓冲区

u8 rx_index = 0; // 接收缓冲区索引，用于指示当前接收到的位置

void usart()
{
    SCON = 0x50; // 设置串口工作模式为方式 1
    TMOD |= 0x20; // 设置定时器 1 工作在方式 2 下
    TH1 = 0xfd; // 波特率设定为 9600
    TL1 = 0xfd;
    ET1 = 0; // 关闭定时器 1 中断
    TR1 = 1; // 启动定时器 1
    EA = 1; // 允许全局中断
    ES = 1; // 允许串口中断
}

void sentdata()
{
    u8 i;
    for (i = 0; i < 12; i++) {
        SBUF = rx_buffer[i]; // 发送缓冲区数据
        while (!TI); // 等待发送完成
        TI = 0; // 清除发送中断标志位
    }
}

void send_s(u8 c)
{
    SBUF = c; // 发送单个字符
    while (!TI); // 等待发送完成
}

```

```

TI = 0;          // 清除发送中断标志位
}

void send_ss(u8 *s)
{
while (*s != '\0') {
send_s(*s); // 逐个发送字符串中的字符
s++;
delay(500); // 延时，用于串口数据稳定
}
}

void ProcessReceivedData(unsigned char *data_d)
{
int valid = 1; // 标记接收数据是否有效
int temp_hour, temp_minute, temp_second;
int temp_year, temp_month, temp_day;
int temp_alarm_hour, temp_alarm_minute;
switch (data_d[0]) {
case 'S': // 设置时间指令
temp_hour    = (data_d[2] - '0') * 10 + (data_d[3] - '0');
temp_minute  = (data_d[5] - '0') * 10 + (data_d[6] - '0');
temp_second  = (data_d[8] - '0') * 10 + (data_d[9] - '0');
if (temp_hour >= 24 || temp_minute >= 60 || temp_second >= 60) {
valid = 0; // 如果时间超出范围，标记为无效
} else {
hour      = temp_hour;
minute    = temp_minute;
second    = temp_second;
}
}
}

```

```

}
break;
case 'D': // 设置闹钟指令
temp_alarm_hour    = (data_d[2] - '0') * 10 + (data_d[3] - '0');
temp_alarm_minute = (data_d[5] - '0') * 10 + (data_d[6] - '0');
if (temp_alarm_hour >= 24 || temp_alarm_minute >= 60) {
valid = 0; // 如果闹钟时间超出范围, 标记为无效
} else {
saved_alarm_hour    = temp_alarm_hour;
saved_alarm_minute = temp_alarm_minute;
}
break;
case 'Q': // 查询日期指令
temp_year = (data_d[2] - '0') * 1000 + (data_d[3] - '0') * 100 +
(data_d[4] - '0') * 10 + (data_d[5] - '0');
temp_month = (data_d[7] - '0') * 10 + (data_d[8] - '0');
temp_day    = (data_d[10] - '0') * 10 + (data_d[11] - '0');
if (temp_year < 1900 || temp_year > 2100 || temp_month < 1 || temp_month >
12 || temp_day < 1 || temp_day > 31 ||
((temp_month == 4 || temp_month == 6 || temp_month == 9 || temp_month ==
11) && temp_day > 30) ||
(temp_month == 2 && temp_day > (is_leap_year(temp_year) ? 29 : 28))) {
valid = 0; // 如果日期超出范围, 标记为无效
} else {
year    = temp_year;
month   = temp_month;
day     = temp_day;

```

```

}
break;
default:
valid = 0; // 其他指令均视为无效
break;
}
if (!valid) {
send_ss("Not in scope\n"); // 发送无效指令提示
}
}
void Serial_ISR() interrupt 4
{
if (RI) {
RI = 0; // 清除接收中断标志位
rx_buffer[rx_index++] = SBUF; // 将接收到的数据存入接收缓冲区
if (rx_buffer[rx_index - 1] == '\n') { // 如果接收到换行符，则表示一条
数据接收完整
sentdata(); // 发送接收到的数据
rx_index = 0; // 重置接收缓冲区索引
ProcessReceivedData(rx_buffer); // 处理接收到的数据
}
}
}
}

```

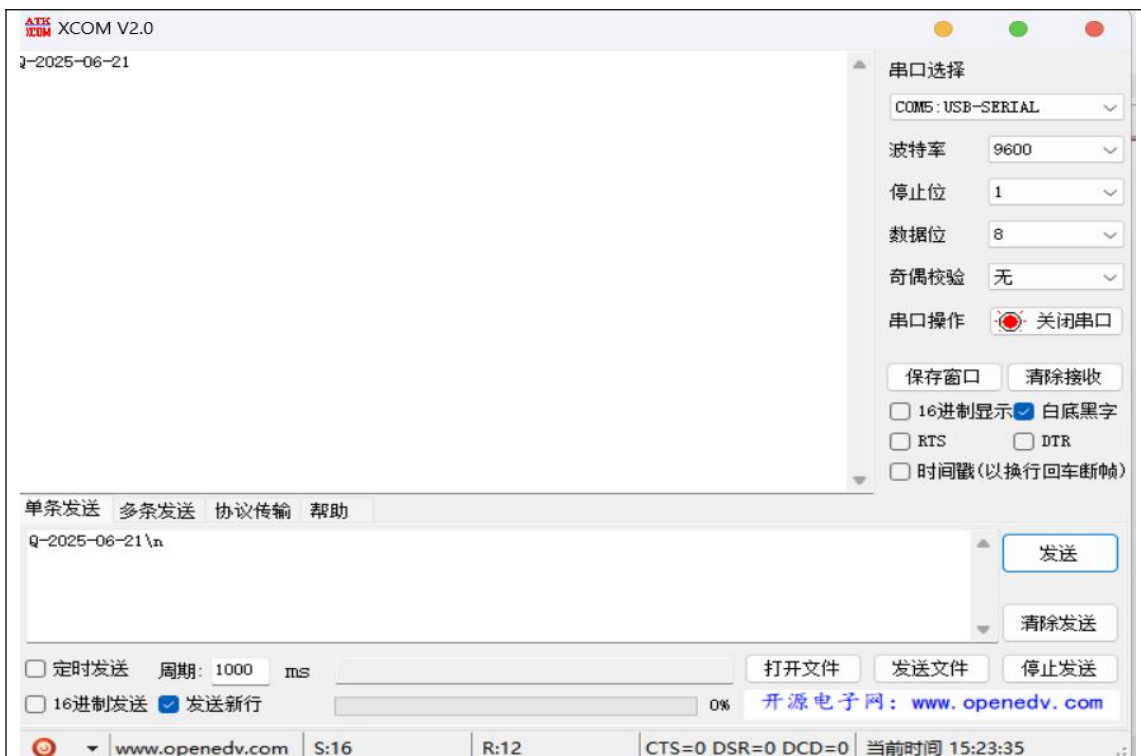



图 24 使用串口助手发送要设置的日期，单片机成功接受到，并将收到的发送了出来



图 25 发现单片机上的时间已设置为串口助手发送的时间



图 26 如果发送的日期超出限制范围，单片机接收到了但不会设置，会给用户提示设置的值超出范围

日期：6 月 27 日

实习（实训）内容：

4.7 利用 Proteus 软件实现系统仿真、系统软硬件调试、实现系统要求功能

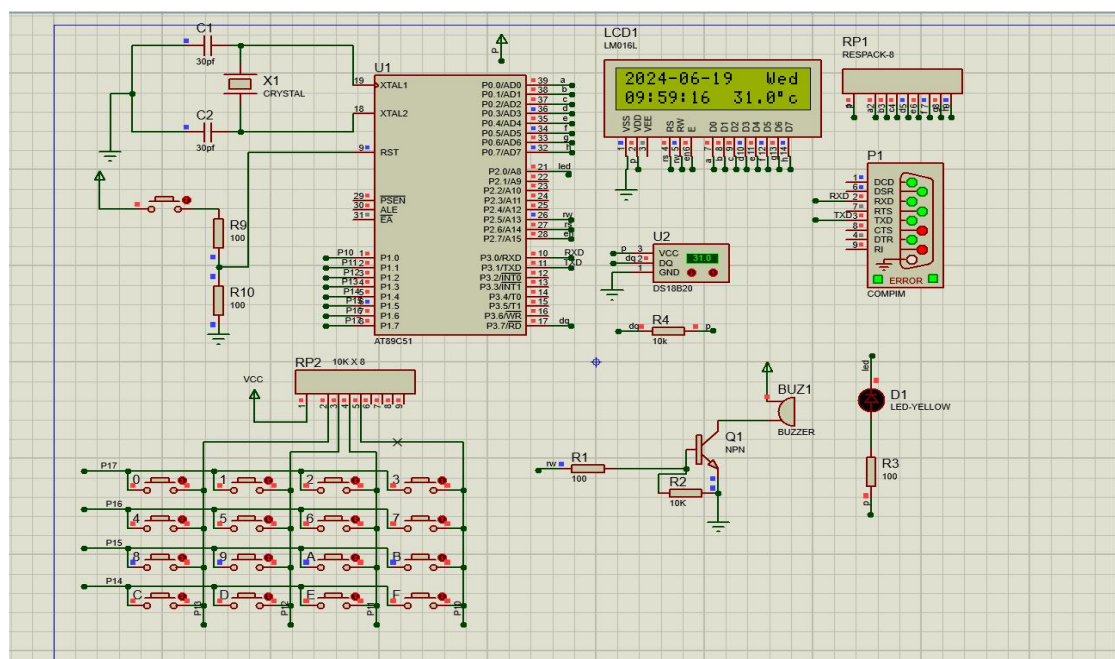


图 27 Proteus 软件实现系统仿真

4.8 主函数设计

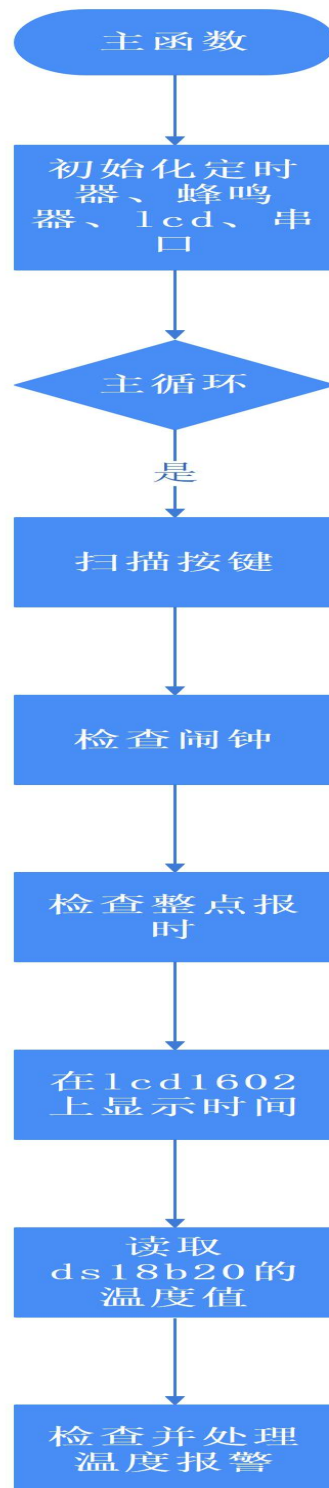


图 28 主函数流程图设计

```

#include "lcd1602.h"
#include "port.h"
#include "beep.h"
#include "ds18b20.h"

void main()
{
    u8 temp_value; // 定义一个无符号 8 位变量，用于存储温度值
    timer0_init(); // 初始化定时器 0
    beep_init();   // 初始化蜂鸣器
    lcd1602_init(); // 初始化 LCD1602 液晶显示屏
    usart();        // 初始化串口通信
    while (1) {
        key_scan(); //
        扫描按键
        AlarmClock(); // 检
        查并处理闹钟事件
        HourlyReporting(); // 检查并处
        理整点报时事件
        temp_value = ds18b20_read_temperature(); // 读取 DS18B20 温
        度传感器的温度值
        TemperatureAlarm(temp_value); // 检查并处理温度报警
        事件
        lcd1602_display_time(); // 在
        LCD1602 上显示当前时间
    }
}

```

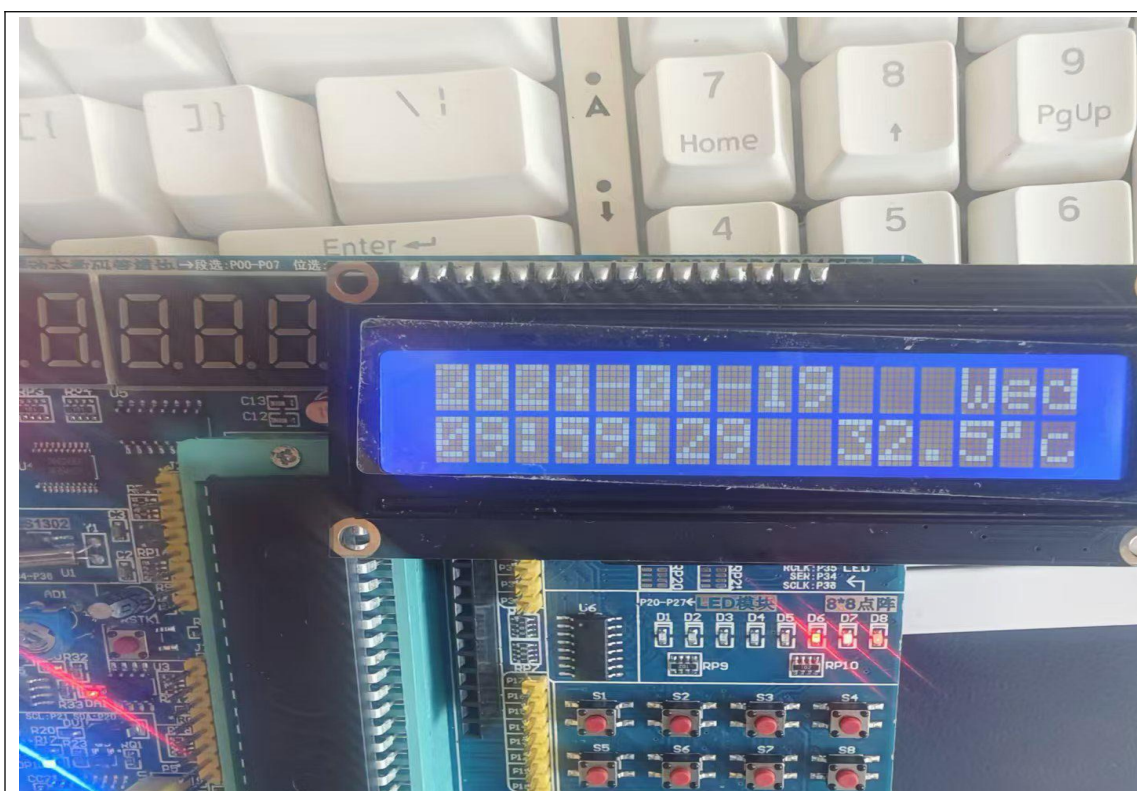



图 29 单片机正常显示

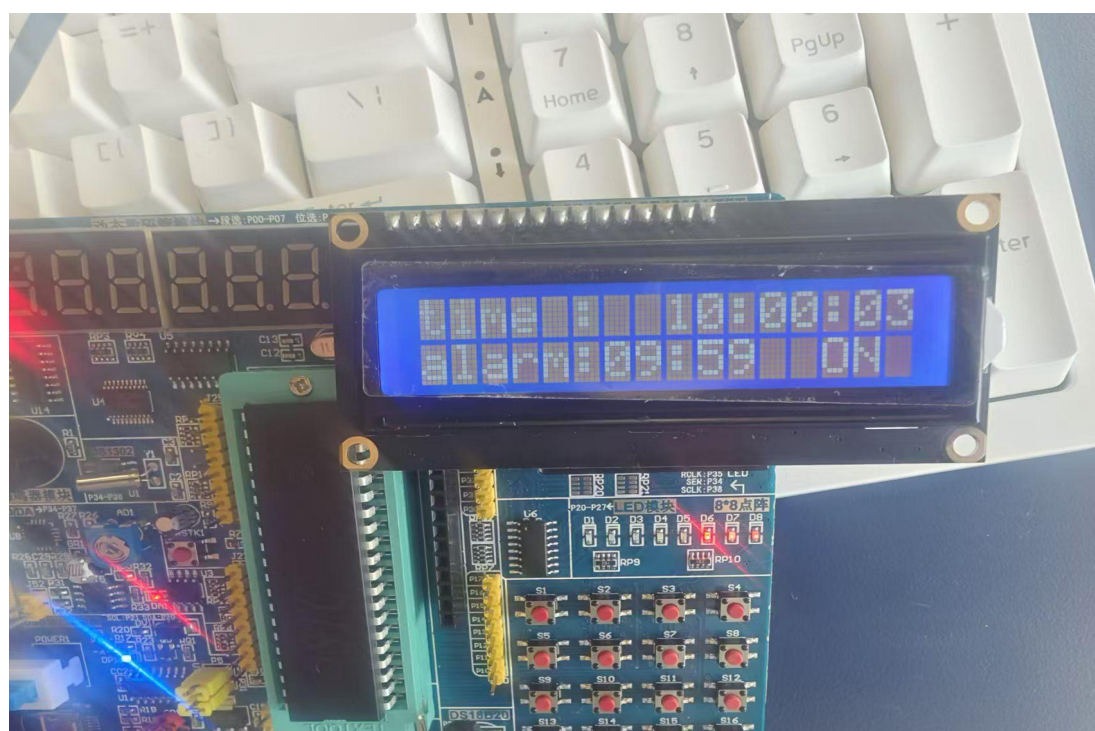


图 30 闹钟界面，闹钟开启状态

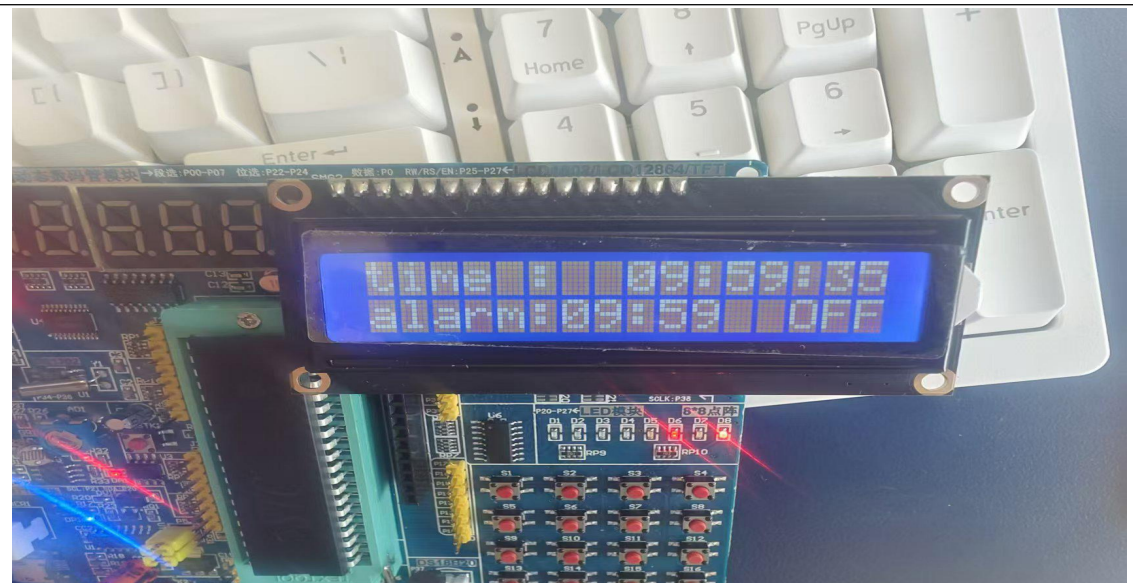


图 31 闹钟界面，闹钟关闭状态

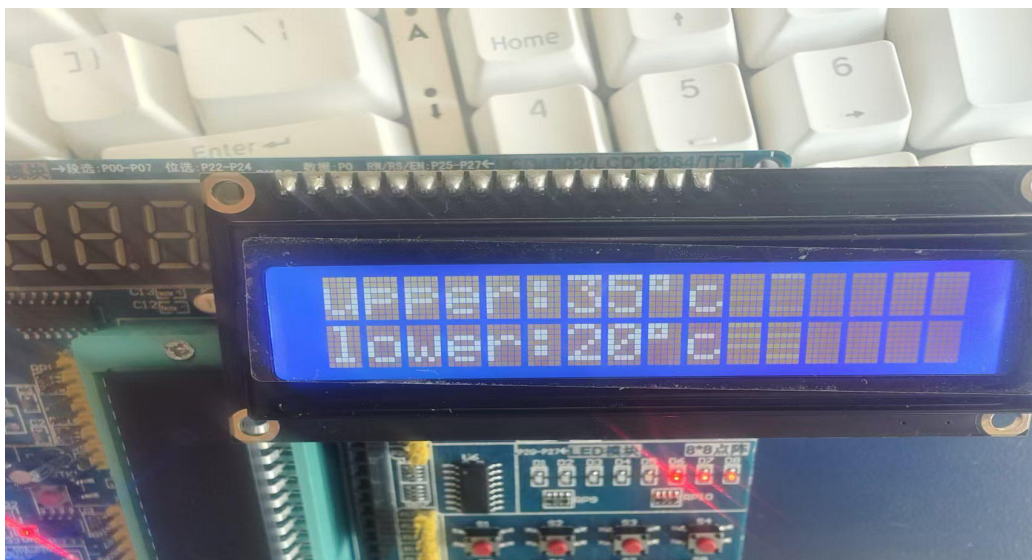


图 32 温度界面，上下限的调节

实习提问：

使用定时器 T1 方式 0，晶振 6Mhz，实现 5s 延迟，中断

```
void timer0_init()
```

```
{
```

```
TMOD = 0x00;
```

```
TH1 = (8192 - 5000) * 6000 / 12 / 256;
```

```
TL1 = (8192 - 5000) * 6000 / 12 % 256;
```

```
ET1 = 1;
```

```

TR1    = 1;

EA      = 1;

}

void timer0_ISR() interrupt 1
{
    static unsigned char count = 0;
    TH1    = (8192 - 5000)*6000/12 / 256;
    TL1    = (8192 - 5000)*6000/12 % 256;
    count++;
    if (count >= 1000)
    {
        count = 0;
        second++; }
    }

Lcd1602 清屏，5*7 点阵 8 位数据接口，光标右移

void lcd1602_write_cmd(u8 cmd)
{
    RS = 0;
    RW = 0;
    EN = 1;
    P0 = cmd;
    delay(1);
    EN = 0;
}

lcd1602_write_cmd(0x01);
lcd1602_write_cmd(0x38);
lcd1602_write_cmd(0x06);

```

实习（实训）总结

实习期间，我参与了一项单片机应用系统的设计与实现任务，这项任务涵盖了多方面的功能需求和技术挑战。首先，项目要求实现精确的时间计时功能，涉及到年、月、日、时、分、秒的准确显示，并将这些数据通过 LCD 液晶屏展示出来。为了实现用户友好的操作界面，我引入了 4×4 矩阵键盘，使得时间的设定和调整变得简单和直观。

除了基本的时间管理功能，项目还包括了闹铃设定和提醒功能的实现。用户可以通过键盘设置闹钟时间，并在设定时间触发时通过 LED 报警模块和液晶屏提示用户。这一功能增强了系统的实用性。在环境感知方面，项目要求集成 DS18B20 数字温度传感器，实时采集周围环境的温度数据，并在 LCD 上进行显示。同时，我通过设定阈值，实现了当环境温度超过或低于设定值时的自动报警功能，LED 指示灯同步闪烁，提醒用户注意环境变化。

在实现这些功能时，我选择了 STC89C52 单片机作为核心控制器，这款单片机以其稳定性和灵活性在嵌入式系统中广泛应用。通过单片机的 I/O 口与 LCD 模块、键盘、温度传感器和 LED 模块进行连接和控制，实现了各功能模块之间的协同工作和信息交换。

在实习过程中，我不仅学到了单片机硬件设计和软件编程的基础知识，还深入理解了如何在实际项目中应用这些知识。在调试过程中，我遇到了各种挑战和问题，例如硬件连接问题、中断处理优化以及代码效率的提升。通过分析问题的根源并采取相应的解决措施，我逐步提升了自己的解决问题的能力和调试技巧。

团队合作也是我在实习中的重要收获。与同事们的交流和协作帮助我更好地理解 and 解决项目中的技术难题，共同努力推动项目进展。在未来，我将继续加强对单片机及其外围设备的学习和掌握，特别是深入学习温度传感器 DS18B20 的原理和使用技巧，以提升项目开发中的技术应用能力。

总结而言，这次实习不仅让我在技术上有了重要的成长和提升，还培养了我解决问题和团队合作的能力。我将继续保持学习的热情，积极参与更多实际项目，不断提升自己在嵌入式系统设计与开发领域的专业能力和实践经验。

实习（实训）成绩评定表

实习（实训）名称		单片机应用系统设计及实现	
序号	考核项目	分值分布	成绩
1	实习考核	30	
2	实习成果	40	
3	实习报告	30	
总分			
指导教师签字			

备注：考核项目及分值分布应依据实习（实训）大纲和实习（实训）内容确定。归档时应归入成绩评定原始记录。

打印稿排版格式

1.正文部分两端对齐，首行缩进 2 字符；左右缩进 0 字符；宋体，小 4 号，不加粗，行距为固定值 25 磅，段前、段后为 0 行。

2.插图须有图号和图名，放在图的下方，居中对齐。

3.表格须有表号和表名，放在表的上方，居中对齐。

4.公式编号，用括号括起来写在右边行末，其间不加虚线。

5.图纸要求：

（1）理工类：图面整洁，布局合理，线条粗细均匀，圆弧连接光滑，尺寸标注规范，文字注释必须使用工程字书写；必须按国家规定标准或工程要求绘制。

（2）艺术类：设计方案要求效果图、结构款式图、平面结构图及面料小样齐备，结构图要绘制精细，要有尺寸数据标注。